

DeepEdu-v1: Efficient and Scalable Agentic LLMs for Vietnamese Education

Quang Nguyen, Hieu Nguyen, Hien Hoang, Toan Pham, Cong Tran, Nam Vu

Posts and Telecommunications Institute of Technology, Ha Noi, Vietnam

Email: {QuangNM.B21CN629, HieuNPT.B23CE031, HienHC.B23CE028, ToanPQ.B23CN835}@stu.ptit.edu.vn, conggt@ptit.edu.vn

Abstract—AI tutoring could markedly improve learning outcomes for students in developing regions such as Vietnam, yet the two obvious paths both fall short. Cloud assistants such as ChatGPT route sensitive student data to foreign servers—violating data-sovereignty laws such as Vietnam’s Decree 53—and, pre-trained on Western-centric corpora, are not organized around the national textbook curriculum, so their knowledge of local content is unsystematic and frequently hallucinated. Self-hosting an open model keeps data on-premise but hits a two-fold wall: post-training quantization (AWQ, GPTQ) tames the static weight footprint, yet the dynamic KV cache and prefill latency of long tutoring contexts still cause out-of-memory failures and slow responses on consumer GPUs, while the model keeps hallucinating on region-specific material.

We present DeepEdu-v1, an AI-tutoring system for Vietnamese education built on SCALE (Self-improving Context-Aware Learning Engine), a framework with two innovations. First, a long-context inference engine amortizes token selection from per-subchunk to per-cluster granularity; on long-context retrieval it issues $7.7\times$ fewer retrieval calls than a state-of-the-art selective-attention baseline, cutting prefill latency (TTFT) by roughly 35% while matching or improving task accuracy. Second, a self-improving agentic layer continuously curates a verified playbook from past interactions instead of fine-tuning, a design intended to progressively reduce reliance on dominant-language priors as trustworthy local knowledge accumulates. In its deployed configuration, DeepEdu achieves a nearly $2\times$ TTFT speedup over standard vLLM serving and lifts agentic accuracy from 70.0% to 79.5% on complex tasks, with the strongest per-track gains across financial-reasoning and interactive-agent benchmarks. Together, these results indicate that scalable, self-improving, curriculum-grounded AI tutoring can be democratized in resource-constrained regions.

Index Terms—Large Language Models, AI Education, Long-Context Inference, Model Quantization, Data Sovereignty, Self-improving Agents

I. INTRODUCTION

Large Language Models (LLMs) have moved beyond text generation into multimodal, tool-augmented agents capable of complex reasoning, planning, and personalized interaction [1]–[3]. In education, this puts within reach something long unavailable in resource-constrained systems: a one-on-one tutor that adapts to each student’s pace and cognitive profile [4]. Our goal is **DeepEdu**: an AI-tutoring platform that gives Vietnamese students a more effective, curriculum-aligned learning path, with LLM agents as its foundation. Realizing this vision, however, runs into two obstacles that a generic chatbot does not solve.

A natural first question is why not simply use a cloud assistant such as ChatGPT. Two reasons rule it out. First, *data sovereignty*: educational records contain highly sensitive personally identifiable information, and routing them to foreign servers directly violates national data-localization laws such as Vietnam’s Decree 53 [5], compelling institutions to self-host open models strictly on-premise. Second, *curriculum grounding*: pre-trained on English-dominant, Western-centric corpora [6], such models are not organized around the Vietnamese textbook curriculum (*Sach Giao Khoa*, SGK); their knowledge of local content is unsystematic and frequently hallucinated [7].¹ A trustworthy tutor must therefore be both self-hosted and explicitly grounded in localized curricular knowledge.

Both requirements are hard on the infrastructure available to public schools. Self-hosting on limited hardware creates a two-fold bottleneck. The static footprint of model weights

¹For instance, early deployments of ChatGPT in Vietnam generated severe historical hallucinations, such as claiming national leaders were founders of the World Health Organization (WHO), or confidently asserting that Emperor Quang Trung and his birth name Nguyen Hue were two distinct kings from opposing dynasties [8]. Such algorithmic hallucinations highlight the severe pedagogical risks of deploying ungrounded models in regional education.

can be tamed by post-training quantization such as AWQ [9] or GPTQ [10]; but as a tutoring session stretches toward a 1M-token context, the *dynamic* KV cache surpasses the static footprint and, together with quadratic prefill latency, causes Out-Of-Memory (OOM) failures and sluggish responses on consumer-grade GPUs—a bottleneck that I/O-aware kernels such as FlashAttention [11] and PagedAttention [12] alleviate but do not remove. Grounding is equally costly the obvious way: continually fine-tuning on evolving curricula is computationally prohibitive and risks catastrophic forgetting [13].

We address both obstacles with **DeepEdu-v1**, built on the **SCALE** framework (Self-improving Context-Aware Learning Engine). SCALE pairs an efficient long-context inference engine—which amortizes selective sparse attention across similar query chunks to keep latency low as context grows—with a self-improving agentic layer that curates a verified, curriculum-grounded playbook from past interactions rather than updating weights. The main contributions of our work are as follows:

- 1) We introduce *Similarity Chunk Rolling*, a long-context inference engine that amortizes selective sparse attention from per-sub-chunk to per-cluster granularity. By issuing a single retrieval per cluster, it reduces retrieval invocations by $7.7\times$ and prefill latency by roughly 35% over a state-of-the-art selective-attention baseline (TokenSelect), while preserving—and on structured-retrieval tasks improving—task accuracy.
- 2) We design a self-improving agentic layer for DeepEdu that, through query-aware context engineering, curates a verified playbook grounding the model in local knowledge and progressively reducing reliance on dominant-language priors—without computationally expensive weight updates.
- 3) We show, across grounded financial-reasoning and interactive-agent benchmarks, that this self-improving loop delivers the strongest per-track gains over a strong agentic baseline, and that in its deployed configuration DeepEdu sustains a nearly $2\times$ TTFT speedup—a step toward democratizing localizable, self-improving AI tutoring in resource-constrained regions under data-sovereignty constraints.

II. RELATED WORK

To contextualize the contributions of DeepEdu, this section surveys recent advancements across the architectural evolution

of LLMs and their application to education, the principal techniques for compressing models and optimizing long-context inference under hardware constraints, the challenges of deploying LLMs on low-resource languages and within localized cultural contexts, and the emerging paradigm of self-improving agentic frameworks.

A. The Evolution of Large Language Models

The landscape of Natural Language Processing (NLP) has been fundamentally transformed by the scaling of neural architectures, passing through several critical milestones. This trajectory is rooted in the Transformer architecture and early scaling laws [14], [15]. Subsequently, the community witnessed the rise of powerful open-weights foundation models that democratized high-performance language modeling [1], [2]. To optimize the intrinsic trade-off between model capacity and inference cost, recent architectures have increasingly adopted the Mixture-of-Experts (MoE) paradigm [16]–[18]. This architectural evolution has recently culminated in highly efficient, post-training reinforced paradigms (such as DeepSeek-R1 and Qwen3) that emphasize advanced logical reasoning and test-time scaling over pure pattern matching [19], [20].

Concurrently, a pivotal paradigm shift has occurred: moving from static sequence-to-sequence generation toward interactive, tool-augmented autonomous agents. This transition was initially catalyzed by models pre-trained on vast codebases, such as Codex [21], which demonstrated that LLMs could translate natural language into actionable programmatic commands. Building upon this, frameworks like ReAct [3] and Toolformer [22] equipped models with the ability to synergize step-by-step reasoning with external API invocations. As reasoning engines matured, the focus shifted toward multi-agent collaboration and fully autonomous software engineering capabilities. Recent advancements, ranging from open-source frameworks like AutoGen and SWE-agent [23], [24] to highly integrated proprietary systems like Claude Code [25], showcase agents capable of navigating complex file systems, executing code iteratively, and autonomously self-correcting. Collectively, these advances established LLM agents as a viable substrate for complex, interactive applications — among which personalized education has emerged as a particularly compelling domain.

B. Foundation Models for Education

The integration of generative AI into educational technology represents a profound shift from static, rule-based Intelligent

Tutoring Systems (ITS) to dynamic, generative frameworks. Early approaches primarily relied on rigid, operation-based formalisms to parse and solve domain-specific tasks, such as mathematical word problems [26]. However, the advent of Large Language Models has fundamentally upgraded this paradigm. Comprehensive surveys now outline the transition toward utilizing foundation models as core cognitive engines for personalized education, emphasizing their capacity to dynamically tailor feedback, simulate roleplay, and adapt to individual learning styles [4].

To bridge the gap between general-purpose text generation and strict pedagogical utility, recent advancements have focused on developing specialized, agentic tutoring frameworks. For instance, models such as MathCoder [27] interleave natural language reasoning with verifiable code execution, ensuring that students receive accurate, step-by-step guidance rather than hallucinated solutions. Furthermore, the integration of LLMs with Knowledge Tracing (KT) paradigms enables these agents to actively model a student’s evolving cognitive state. By maintaining dynamic representations of a learner’s historical performance, state-of-the-art educational agents can autonomously adapt their pedagogical scaffolding—seamlessly shifting between direct hints and deeper Socratic questioning based on real-time comprehension [28]. Nevertheless, the practical deployment of such educational agents at scale remains constrained by two orthogonal challenges that have received comparatively less attention: the computational cost of running these models on accessible hardware, and their tendency to misalign with localized curricula — issues we now examine in turn.

C. Post-Training Quantization for Efficient Deployment

A fundamental prerequisite for deploying state-of-the-art LLMs on consumer-grade hardware is reducing the static memory footprint of model weights. Post-training quantization (PTQ) has emerged as the dominant solution, compressing pre-trained weights into low-bit representations without expensive re-training. GPTQ [10] pioneered accurate one-shot quantization for generative transformers, employing an approximate second-order procedure to minimize layer-wise reconstruction error at 3 or 4 bits. AWQ [9] further observes that weight importance is highly non-uniform: a small fraction of salient weights, identified through activation magnitudes, disproportionately influences output quality, and selectively preserving them through per-channel scaling achieves superior accuracy under

hardware-friendly uniform bit-widths. However, quantization addresses only the static weight footprint; the dynamic memory growth of the KV cache during long-context inference remains entirely unresolved, motivating the complementary techniques surveyed next.

D. Knowledge Distillation for Compact Educational Models

An alternative compression paradigm transfers the capabilities of large teacher models into smaller student architectures through knowledge distillation (KD). Traditional KD [29] matches soft probability distributions, but often induces hallucinations in generative tasks due to the mode-covering behavior of forward Kullback-Leibler divergence. MiniLLM [30] mitigates this through reverse KLD with policy gradient optimization, forcing students to concentrate probability mass on high-quality generation regions. Beyond probability matching, Hsieh et al. [31] introduced step-by-step distillation that extracts intermediate reasoning trajectories from proprietary teachers, enabling compact students to internalize multi-step problem-solving heuristics — a capability particularly valuable for pedagogical scaffolding in educational tutoring.

Nevertheless, distillation pipelines remain computationally intensive at training time and produce student models with static knowledge that cannot adapt to evolving curricula without repeated re-distillation, limiting their applicability for institutions facing both hardware and content-update constraints.

E. Long-Context Inference Optimization

Due to the quadratic computational complexity of standard attention, Transformer-based LLMs historically operate within limited pre-training context windows. Extending this window has been pursued through three complementary directions. First, modifications to positional encoding via interpolation and rotary embeddings enable zero-shot length generalization without re-training [32]–[36]. Second, long-context post-training explicitly adapts model weights to extended sequences, often supported by sequence parallelism techniques [37]–[40]. Third, the integration of specialized memory modules enables processing of effectively unbounded sequences [41]–[44].

However, even with these extensions, long-context inference suffers from prohibitive latency during the prefilling stage. System-level optimizations such as IO-aware FlashAttention and PagedAttention mitigate GPU memory bottlenecks but do not reduce the fundamental computational complexity of attention [11], [12]. To process million-token contexts

efficiently, a growing body of work exploits attention sparsity through input-independent patterns such as localized windows [45], [46] and attention sinks that preserve critical initial tokens [47], [48]. While effective, these heuristic patterns can permanently discard informative tokens, motivating dynamic KV cache selection methods that adaptively retain relevant entries [49], [50]. State-of-the-art frameworks further refine selection granularity, progressing from block-level memory units [51], through query-aware page-level sparsity [52] and dynamic context selection [53], [54], to fine-grained token-level selection that bypasses block-level inefficiencies entirely [55].

While token-level methods such as TokenSelect substantially reduce per-step attention cost, the interaction between selective sparse attention and chunked prefill at very long contexts remains an active area of investigation.

F. LLMs on Low-Resource Languages

While modern LLMs exhibit remarkable zero-shot and few-shot reasoning capabilities in high-resource languages like English, their performance often degrades significantly when applied to low-resource languages (LRLs) such as Vietnamese. This degradation is primarily attributed to a severe underrepresentation of LRL tokens in the pre-training corpora, which limits the models’ native linguistic alignment.

To circumvent this linguistic bottleneck without resorting to computationally prohibitive pre-training from scratch, recent literature has explored several efficient adaptation strategies. At the inference level, Nguyen et al. [56] propose leveraging the models’ dominant English capabilities through linguistically-diverse prompts. This approach acts as a cross-lingual bridge, effectively anchoring LRL inputs to the models’ highly structured English latent space to improve zero-shot generalization. Complementarily, Cahyawijaya et al. [6] demonstrate that LLMs possess latent cross-lingual capabilities that can be explicitly activated via In-Context Learning (ICL). Their empirical findings show that providing a limited number of task-specific demonstrations in the target language allows the model to dynamically adapt its reasoning pathways to the LRL without requiring any parameter updates.

For applications requiring deeper structural alignment, prompt engineering alone is often insufficient. To address this, Nag et al. [13] introduce efficient continual pre-training (CPT) techniques tailored for LRLs. Their methodology natively injects new linguistic knowledge into the foundation model’s weights while actively mitigating the catastrophic forgetting of

general reasoning capabilities, all under strict computational constraints. Collectively, these methodologies highlight that effective LRL adaptation can be achieved without invasive parameter updates, particularly when combined with carefully curated in-context signals.

G. Hallucination in Localized and Cultural Contexts

While modern LLMs demonstrate strong generalization capabilities, their deployment in region-specific applications is frequently hindered by factual hallucinations. Ji et al. [57] identify that hallucinations fundamentally stem from data divergence, where models learn spurious correlations from imbalanced pre-training corpora. This issue is severely amplified by the Western-centric nature of foundational models. As Huang et al. [58] highlight, when LLMs process queries outside their dominant data distribution, they often generate confident confabulations rather than acknowledging knowledge boundaries. Empirical evaluations by Bang et al. [59] confirm this limitation, demonstrating that LLMs exhibit significantly higher hallucination rates and degraded reasoning when applied to low-resource languages and non-Western cultural contexts. In educational settings, relying on such ungrounded models introduces critical pedagogical risks. These findings establish that scaling pre-training alone cannot resolve the localization gap, particularly in educational deployments where factual accuracy on regional content is non-negotiable. This persistent gap motivates runtime grounding mechanisms capable of injecting localized knowledge without the prohibitive cost of continual re-training.

H. Agentic Systems and Self-Evolving Context

The rapid evolution of LLMs has introduced a paradox in knowledge management: while models store vast generalized knowledge within their parametric memory, their ability to seamlessly integrate deeply specialized domain heuristics remains limited. Traditional fine-tuning is rigid, computationally expensive, and susceptible to catastrophic forgetting. Conversely, current prompt optimization methods often suffer from brevity bias — the tendency to compress prompts into short, generic instructions that strip away essential technical nuances required for high-precision tasks [60]. Furthermore, attempts at monolithic rewriting in long-context windows frequently trigger context collapse, wherein iterative LLM rewrites progressively erode accumulated domain knowledge into uninformative summaries.

TABLE I: Positioning of DeepEdu against representative prior directions. ✓: directly addressed; (✓): partial; -: not addressed. Columns: on-premise/data-sovereignty fit, long-context efficiency, self-improvement without fine-tuning, and localization.

Direction	On-prem.	Long-ctx	Self-impr.	Local.
Quantization [9], [10]	✓	–	–	–
Sparse long-context [51], [55]	–	✓	–	–
Agentic context [62], [65]	(✓)	–	✓	–
LRL adaptation [6], [13]	–	–	–	✓
DeepEdu (ours)	✓	✓	✓	✓

To overcome these barriers and build agentic systems that self-improve without weight updates, recent literature explores natural language feedback and verbal reinforcement learning [61]–[63]. Techniques leveraging reflective prompt evolution [64] and agentic context engineering [65] effectively transform the static context into a living, continuously updated "playbook." These methodologies enable LLM agents to adapt and fine-tune their behavior exclusively through dynamic context memory, preserving granular domain insights without the massive overhead of standard fine-tuning [66].

I. Discussion and Positioning

Table I summarizes where DeepEdu sits relative to the four directions above. Each addresses one facet of on-premise educational deployment but leaves the others open: quantization shrinks static weights yet does nothing for the dynamic long-context cost; selective sparse attention accelerates long contexts but neither self-improves nor localizes; agentic context engineering self-improves without weight updates but is not built for million-token efficiency or a specific curriculum; and low-resource-language adaptation localizes, but through weight updates that are costly to maintain as curricula evolve. DeepEdu is, to our knowledge, the first framework to occupy their intersection: it couples a cluster-level long-context engine with a self-improving, curriculum-grounded agentic layer, so that a single on-premise system is efficient, self-improving, and localizable at once.

III. PRELIMINARIES

This section establishes the notation and the two paradigms our framework builds on. We first summarize Transformer inference and Multi-Head Attention (§III-A) and formalize long-context inference as a Selective Sparse Attention problem (§III-B)—the basis of our inference engine—then define

Agentic Context Engineering (§III-C), the paradigm underlying our self-improving agentic layer.

A. LLM Inference and Multi-Head Attention

a) Notation: Throughout this paper, we adopt the following unified notation. Let d denote the model hidden dimension, H the number of attention heads, and $d_h = d/H$ the per-head dimension. For a transformer layer processing n input tokens, we denote its hidden states by $\mathbf{X} \in \mathbb{R}^{n \times d}$. The KV Cache accumulated from previously processed tokens has length N , and we use $h \in \{1, \dots, H\}$ to index attention heads.

b) Multi-head attention: Given hidden states $\mathbf{X}$, each head h projects $\mathbf{X}$ into per-head queries, keys, and values $\mathbf{Q}^{(h)}, \mathbf{K}^{(h)}, \mathbf{V}^{(h)} \in \mathbb{R}^{n \times d_h}$ and computes scaled dot-product attention (SDPA) [14]:

$$\mathbf{O}^{(h)} = \text{softmax}\left(\frac{\mathbf{Q}^{(h)}(\mathbf{K}^{(h)})^\top}{\sqrt{d_h}}\right) \mathbf{V}^{(h)}, \quad (1)$$

whose per-head outputs are concatenated and projected back to dimension d . The inner softmax in Equation (1) has quadratic complexity $\mathcal{O}(n^2 d_h)$ in sequence length—the principal bottleneck of long-context inference.

c) Prefill, decode, and the KV Cache: Inference has two stages. *Prefill* processes the whole prompt of length n_{in} in one pass and persists the per-head keys and values as a KV Cache $\mathbf{K}_{\text{cache}}^{(h)}, \mathbf{V}_{\text{cache}}^{(h)} \in \mathbb{R}^{N \times d_h}$ (initially $N = n_{in}$); it is compute-bound and dominates latency for long inputs. *Decode* then generates one token per pass, appending its key and value to the cache so that N grows by one each step. Each decode step is light, but the KV Cache footprint grows linearly with context, straining consumer-grade hardware.

d) Two-level chunking for long-context prefill: To bound peak memory consumption during prefill and enable processing of inputs that exceed available VRAM, modern inference frameworks such as vLLM [12] and SGLang adopt chunked prefill: the input sequence is partitioned into consecutive outer chunks, each of length at most L_{outer} tokens (typically $L_{\text{outer}} = 8192$). Each outer chunk is processed in a separate forward pass; the last outer chunk may be shorter when n_{in} is not a multiple of L_{outer} . This outer chunking caps the peak activation memory of any single forward pass but does not, by itself, reduce the cost of attending to the growing KV Cache: a naive implementation still incurs $\mathcal{O}(\ell \cdot N \cdot d_h)$ attention cost per outer chunk under Equation (1), where $\ell \leq L_{\text{outer}}$ denotes the actual length of the current outer chunk and N is the cumulative KV Cache length.

To further reduce per-step attention cost, recent selective sparse attention methods such as TokenSelect [55] introduce a second, finer-grained level of partitioning *within* each outer chunk. Specifically, the queries of an outer chunk of length ℓ are subdivided into $M = \lceil \ell/L \rceil$ sub-chunks, denoted $\{\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_M\}$. Each sub-chunk has length L (typically $L = 512$), except possibly the last, which has length $\ell - (M - 1)L \leq L$. For each sub-chunk, the method invokes a query-aware selection function to identify a small subset of critical KV Cache entries, then computes sparse attention restricted to that subset. Crucially, this selection-then-attention procedure is applied sequentially to each sub-chunk within the outer chunk: M independent retrieval calls followed by M independent attention calls per outer chunk. While this scheme effectively bypasses the quadratic attention cost, the sequential nature of the inner loop exposes a substantial opportunity for amortization when consecutive sub-chunks share similar query representations — a redundancy our work directly targets.

B. Selective Sparse Attention

The quadratic cost of SDPA in Equation (1), combined with the abundant sparsity empirically observed in LLM attention maps [51], [52], [55], motivates the selective sparse attention (SSA) paradigm: rather than attending to the entire KV Cache, only a small subset of critical tokens is dynamically selected for each query. Our inference engine builds directly on this paradigm; we restate it below in our notation, following the formulation of TokenSelect [55].

Definition 1 (Selective Sparse Attention). *Consider a transformer layer at any inference step, with per-head queries $\mathbf{Q}^{(h)} \in \mathbb{R}^{C \times d_h}$ for C current input tokens ($C = 1$ during decoding, $C \leq L$ for one sub-chunk during prefill where L is the sub-chunk size defined in §III-A), and a KV Cache $\mathbf{K}_{\text{cache}}^{(h)}, \mathbf{V}_{\text{cache}}^{(h)} \in \mathbb{R}^{N \times d_h}$. The full attention output following Equation (1) is:*

$$\mathbf{O}^{(h)} = \text{softmax} \left(\frac{\mathbf{Q}^{(h)} (\mathbf{K}_{\text{cache}}^{(h)})^\top}{\sqrt{d_h}} \right) \mathbf{V}_{\text{cache}}^{(h)}. \quad (2)$$

Building on Equation (2), selective sparse attention restricts the softmax to a subset of $k \ll N$ cache entries, yielding the sparse approximation $\hat{\mathbf{O}}^{(h)}$:

$$\hat{\mathbf{O}}^{(h)} = \text{softmax} \left(\frac{\mathbf{Q}^{(h)} (\mathbf{K}_{\text{select}}^{(h)})^\top}{\sqrt{d_h}} \right) \mathbf{V}_{\text{select}}^{(h)}, \quad (3)$$

The two formulations differ only in which cache rows enter the

softmax: the selected matrices $\mathbf{K}_{\text{select}}^{(h)}, \mathbf{V}_{\text{select}}^{(h)} \in \mathbb{R}^{k \times d_h}$ index $\mathbf{K}_{\text{cache}}^{(h)}$ and $\mathbf{V}_{\text{cache}}^{(h)}$ at the positions $\mathcal{I}$ chosen by a selection function $\mathcal{S}$:

$$\mathcal{I} = \mathcal{S}(\mathbf{Q}, \mathbf{K}_{\text{cache}}), \quad \mathcal{I} \subseteq \{1, \dots, N\}, \quad |\mathcal{I}| = k. \quad (4)$$

Here $\mathbf{Q}$ and $\mathbf{K}_{\text{cache}}$ denote the multi-head queries and cached keys aggregated across all heads, allowing $\mathcal{S}$ to leverage cross-head information when scoring token criticality. The design objective is to construct $\mathcal{S}$ such that the approximation gap $\|\mathbf{O}^{(h)} - \hat{\mathbf{O}}^{(h)}\|_2^2$ is minimized while keeping k within the hardware budget.

Selection functions $\mathcal{S}$ range from fixed patterns (attention sinks and recent tokens), through query-independent scoring, to query-aware token-level selection (surveyed in §II-E). Our engine builds on the query-aware token-level scheme of TokenSelect [55], which conditions $\mathcal{S}$ on the current query $\mathbf{Q}$ for high approximation quality, at the cost of a retrieval call per step.

a) Sub-chunk redundancy in selective sparse attention:

Together, Equations (3)–(4) define a single *selection-then-attention* step, the unit our inference engine reorganizes. A critical observation underexplored by prior work is that, under the two-level chunking scheme of §III-A, this step—and in particular the costly selection function $\mathcal{S}$ —is invoked independently for each of the M inner sub-chunks $\{\mathbf{X}_1, \dots, \mathbf{X}_M\}$ that constitute a single outer prefill chunk. Each invocation triggers a full pass over the growing KV Cache to score and retrieve the top- k critical tokens, followed by an attention call on the selected subset. When consecutive sub-chunks exhibit highly similar query representations — as is often the case within a coherent passage of text — a substantial fraction of their retrieved index sets overlap, yet the retrieval cost is incurred in full for each sub-chunk independently. This per-sub-chunk invocation pattern induces redundant retrieval computation that grows linearly with M , a phenomenon we empirically analyze in Section IV and address through our proposed *cluster-level* selection paradigm in Section V, which groups similar consecutive sub-chunks before invoking $\mathcal{S}$ a single time per cluster.

C. Agentic Context Engineering

While selective sparse attention addresses the computational bottleneck, the semantic bottleneck—grounding the model in localized, trustworthy knowledge—calls for a complementary

paradigm. Our agentic layer builds on Agentic Context Engineering (ACE) [65], which adapts an LLM agent by editing its context rather than its weights. We restate it in the notation used later.

Definition 2 (Agentic Context Engineering). *Let a backbone LLM with fixed parameters ϕ process a stream of interactions $\mathcal{D} = \{(q_t, c_t, y_t)\}_{t=1}^T$, where q_t is a query, c_t its available context, and y_t a feedback signal (e.g., a reference answer or a deterministic environment outcome). ACE maintains a structured playbook $\mathcal{P}_t$ of itemized natural-language entries and, after each interaction, applies a validated set of edits Δ_t distilled from the resulting trajectory:*

$$\mathcal{P}_{t+1} = \text{Apply}(\mathcal{P}_t, \Delta_t), \quad \phi_{t+1} = \phi_t. \quad (5)$$

Adaptation therefore occurs entirely in inspectable context, leaving the weights ϕ unchanged.

Equation (5) leaves open two design choices that decide whether such an agent actually improves: *how* the edits Δ_t are produced, and *which* entries condition each interaction as the playbook grows. Sections IV and V resolve these in turn—the former through empirical observations, the latter through the concrete Generator–Reflector–Curator design of our agentic layer.

IV. MOTIVATIONS AND OBSERVATIONS

The design of SCALE is grounded in five empirical observations, three concerning the long-context engine and two concerning the agentic layer. First, consecutive query sub-chunks within an outer chunk exhibit high mutual similarity across the depth of the model (§IV-A). Second, leveraging this similarity through a tunable threshold θ enables aggressive cluster formation, with a controllable trade-off between amortization and selection granularity (§IV-B). Third, the top- k indices retrieved by a single cluster-level query closely match those that would be retrieved per-sub-chunk, confirming that cluster-level amortization preserves selection fidelity (§IV-C). Fourth, as the Playbook grows, supplying each Generator call with only the query-relevant rules is both faster and more accurate than passing the entire Playbook, and a length-matched random control confirms the gain comes from query–rule matching rather than from shorter prompts alone (§IV-D). Fifth, auditing the agent’s incorrect trajectories reveals that its failures are not isolated accidents but recur across superficially different tasks, so the same underlying weaknesses resurface until they

are explicitly addressed (§IV-E). Together, these observations directly motivate the cluster-level sparse selection of our long-context engine (§V-A) and the playbook-evolving design of our agentic layer (§V-B).

A. Consecutive Sub-Chunk Similarity within Outer Chunks

Under the chunked prefill scheme of §III-A, each outer chunk of length ℓ is subdivided into $M = \lceil \ell/L \rceil$ sub-chunks $\{\mathbf{X}_1, \dots, \mathbf{X}_M\}$ with $L = 512$. We empirically investigate whether the query representations of these sub-chunks remain similar enough across $i = 1, \dots, M-1$ to justify amortizing retrieval across them. Following TokenSelect [55], we summarize each sub-chunk $\mathbf{X}_i$ by its mean-pooled query representation:

$$\mathbf{v}_i = \frac{1}{|\mathbf{X}_i|} \sum_{t=1}^{|\mathbf{X}_i|} \mathbf{q}_{i,t}, \quad (6)$$

where $\mathbf{q}_{i,t} \in \mathbb{R}^d$ denotes the multi-head query of the t -th token in sub-chunk $\mathbf{X}_i$, so that $\mathbf{v}_i \in \mathbb{R}^d$; we write $\mathbf{v}_i^{(h)} \in \mathbb{R}^{d_h}$ for its head- h slice. The pairwise similarity between consecutive sub-chunks is measured by cosine similarity $\cos(\mathbf{v}_i, \mathbf{v}_{i+1})$.

To probe this, we profile prefill over six representative long-context tasks from InfiniteBench [67] (Code.D, R.KV, En.Dia, Math.F, R.Num, and R.PK) using Qwen2-7B-Instruct [16] as the backbone ($L_{\text{outer}} = 8192$, $L = 512$), recording $\cos(\mathbf{v}_i, \mathbf{v}_{i+1})$ for every consecutive sub-chunk pair across five layers that span the depth of the model: shallow (Layers 0 and 6), middle (Layers 13 and 20), and deep (Layer 27).

Table II summarizes the resulting distribution, and the picture is unambiguous: consecutive sub-chunks remain highly similar throughout the network. The median cosine similarity stays above 0.92 at every layer—exceeding 0.94 at the shallow and deep layers—and even the 10th percentile never falls below 0.86, indicating that high similarity is the prevailing regime rather than an artifact of a few dominant pairs. Mass concentrates at the upper end of the distribution and thins only at the extreme lower tail, where a small number of transitions drop sharply, down to 0.33 at Layer 13. We attribute these rare drops to genuine semantic shifts—such as boundaries between distinct documents or context segments—rather than pervasive noise, an interpretation corroborated in §IV-B, where our anchor-based clustering closes a cluster precisely at such transitions.

This concentration of mass at high similarity suggests that treating each sub-chunk as an independent retrieval target is largely redundant: the query representations driving these

TABLE II: Distribution of consecutive sub-chunk cosine similarity $\cos(\mathbf{v}_i, \mathbf{v}_{i+1})$, aggregated across six long-context tasks (Qwen2-7B-Instruct, $L = 512$). Consecutive sub-chunks share nearly identical query representations at every layer, with the low-similarity tail confined to rare semantic boundaries.

Layer	Mean	Min	Max	P10	Median	P90
Layer 0	0.938	0.442	1.000	0.887	0.943	0.982
Layer 6	0.956	0.797	1.000	0.934	0.958	0.977
Layer 13	0.925	0.329	0.995	0.876	0.934	0.969
Layer 20	0.918	0.580	0.996	0.868	0.924	0.963
Layer 27	0.943	0.434	1.000	0.870	0.959	0.987

retrievals are nearly identical for most consecutive pairs. Two design questions follow: *how aggressively* can we group similar sub-chunks into clusters, and *at what cost* to selection fidelity? We address these questions in turn in the next two subsections.

B. Cluster Formation Under Tunable Similarity Threshold

The high mutual similarity established in §IV-A suggests that consecutive sub-chunks can be grouped into clusters whenever their pairwise similarity exceeds a threshold θ . We empirically characterize how the choice of θ controls the aggressiveness of cluster formation across the depth of the model, exposing a tunable accuracy–efficiency trade-off.

To this end, for each outer chunk processed during prefill we apply the anchor-based clustering described in §V—consecutive sub-chunks join a cluster as long as their similarity to the cluster anchor exceeds θ , up to a maximum cluster size of $L_{\max} = 8 \times L = 4096$ tokens—and record the resulting cluster sizes as θ is swept over $\{0.95, 0.97, 0.99, 0.999\}$.

Table III reports the mean cluster size at each threshold. The mechanism is markedly *depth-dependent*: shallow layers (Layers 0 and 6) and the final layer (Layer 27) sustain large clusters across a wide range of θ , whereas the middle layers (Layers 13 and 20) fragment far more readily as θ tightens—at $\theta = 0.99$, for instance, the mean cluster at Layer 20 collapses to 1628 tokens while Layer 0 remains near-maximal at 4013. This asymmetry is consistent with prior findings that mid-depth layers in autoregressive LLMs encode richer semantic representations than either shallow or final layers [68]: the more discriminative the representation, the more readily our mechanism detects a genuine semantic transition and closes the cluster.

Zooming in on Layer 20, the most threshold-sensitive layer, Table IV reports the fraction of clustering decisions that yield cluster sizes of 1, 2, $\dots$, 8 blocks. At the default operating point $\theta = 0.95$, more than 94% of clustering decisions reach

TABLE III: Mean cluster size (in tokens) at each threshold θ . Higher θ produces smaller clusters as the mechanism increasingly detects semantic boundaries; the effect is most pronounced at middle layers where semantic discrimination is highest. Maximum cluster size is $L_{\max} = 4096$.

Layer	$\theta = 0.95$	$\theta = 0.97$	$\theta = 0.99$	$\theta = 0.999$
Layer 0	4025	4025	4013	3446
Layer 6	4025	4021	3904	1616
Layer 13	3988	3699	1947	829
Layer 20	3964	3260	1628	694
Layer 27	4017	3992	3695	1509

TABLE IV: Distribution of cluster sizes (in L -blocks) at Layer 20 under varying threshold θ . At $\theta = 0.95$, 94% of clusters reach the maximum size of 8 blocks; at $\theta = 0.999$, the mechanism reverts to single-block selection 83% of the time.

Cluster size	$\theta = 0.95$	$\theta = 0.97$	$\theta = 0.99$	$\theta = 0.999$
1 block	0.6%	2.4%	36.7%	82.6%
2 blocks	0.9%	7.2%	19.2%	9.2%
3 blocks	1.4%	8.2%	11.0%	3.7%
4 blocks	1.5%	8.0%	8.2%	2.1%
5–7 blocks	1.3%	12.4%	9.3%	2.0%
8 blocks (max)	94.2%	61.7%	15.5%	0.5%

the maximum cluster size of 8 blocks, reducing the number of retrieval invocations of $\mathcal{S}$ per outer chunk by approximately $8\times$ relative to the per-sub-chunk baseline. As θ tightens to 0.999, the mechanism gradually reverts to fine-grained per-sub-chunk selection, with single-block decisions dominating (82.6%). This continuous spectrum exposes θ as a principled tuning knob: lower θ maximizes amortization, higher θ maximizes selection granularity.

These results establish that aggressive cluster formation is *achievable* under realistic operating conditions: at $\theta = 0.95$, the dominant operating point of our mechanism, the overwhelming majority of clustering decisions yield maximum-size clusters. The remaining question is whether this aggressive amortization preserves the fidelity of the underlying selection function $\mathcal{S}$, which we examine next.

C. Top- k Overlap Under Cluster-Level Retrieval

The first two observations established that consecutive sub-chunks are highly similar (§IV-A) and that this similarity enables aggressive cluster formation (§IV-B). We now verify the critical remaining question: when the cluster’s anchor query is used in place of per-sub-chunk queries, does the resulting top- k retrieval recover the same critical KV Cache entries as the per-sub-chunk baseline?

TABLE V: Top- k overlap ratio ρ (%) between the anchor query and per-sub-chunk retrieval at varying similarity thresholds θ , with $k = 8192$. Higher θ yields tighter overlap, confirming that anchor-driven retrieval faithfully approximates per-sub-chunk retrieval in the operating regime of our mechanism.

θ	Min	Median	Mean	Max
0.95	88.2	97.9	96.8	100.0
0.97	90.5	98.6	97.7	100.0
0.99	93.1	99.3	98.6	100.0

To answer this, for each cluster $\mathcal{C} = \{\mathbf{X}_i, \mathbf{X}_{i+1}, \dots, \mathbf{X}_{i+|\mathcal{C}|-1}\}$ formed under threshold θ , we perform two retrievals against the same KV Cache:

- **Per-sub-chunk:** retrieve the top- k index sets $\mathcal{I}_i, \mathcal{I}_{i+1}, \dots, \mathcal{I}_{i+|\mathcal{C}|-1}$ independently using $\mathbf{v}_i, \mathbf{v}_{i+1}, \dots$, then take their union $\mathcal{I}_{\text{union}} = \bigcup_j \mathcal{I}_{i+j}$.
- **Anchor (cluster-level):** retrieve a single top- k index set $\mathcal{I}_{\mathcal{C}}$ using only the *anchor fingerprint* $\mathbf{v}_i$ —the query representation of the first sub-chunk, against which all other members were admitted—shared across the whole cluster, exactly as SCR does at inference time (§V-A3).

We measure the *overlap ratio*:

$$\rho(\mathcal{C}) = \frac{|\mathcal{I}_{\mathcal{C}} \cap \mathcal{I}_{\text{union}}|}{|\mathcal{I}_{\mathcal{C}}|}, \quad (7)$$

which quantifies the fraction of anchor-retrieved indices that would also have been retrieved by independent per-sub-chunk retrievals. We sweep $\theta \in \{0.95, 0.97, 0.99\}$, matching the operating points of §IV-B, with $k = 8192$.

As reported in Table V, the overlap is consistently high and rises monotonically with θ : the median ρ already reaches 97.9% at $\theta = 0.95$ and exceeds 99% at $\theta = 0.99$, while the minimum stays above 88% across all measured clusters. In other words, throughout the operating range of our mechanism a single anchor-driven retrieval recovers nearly all of the critical tokens that per-sub-chunk retrieval would have selected independently, rendering the $|\mathcal{C}|$ separate retrievals largely redundant.

The three observations together establish a chain of evidence for the cluster-level paradigm. The high consecutive similarity (§IV-A) creates the opportunity; the aggressive cluster formation at $\theta = 0.95$ (§IV-B) demonstrates that the opportunity is realizable in practice; and the high top- k overlap (§IV-C) confirms that amortization does not compromise selection fidelity. Our long-context inference engine (§V) operationalizes these insights through the Similarity Chunk Rolling mechanism, adopting $\theta = 0.95$ as the default operating point.

D. Query-Aware Retrieval for Efficient Playbook Execution

As ACE accumulates more rules, supplying the entire Playbook to every Generator call makes execution increasingly expensive and exposes the model to guidance that is unrelated to the current task. The relevant question is therefore not only whether a shorter prompt is faster, but whether the rules retained in that prompt are the ones needed for the current query.

We evaluate a fixed, fully-curated Formula Playbook in eval only mode (distinct from the un-adapted Origin Playbook of Section VI-B) with the same Qwen3-4B-Instruct-2507 [20] backbone, decoding configuration, and 200 test examples in all conditions. The full-Playbook condition supplies every entry to the Generator. Semantic RAE supplies the top $K = 10$ entries most relevant to the query and context. As a length-matched control, random retrieval supplies ten randomly selected entries.

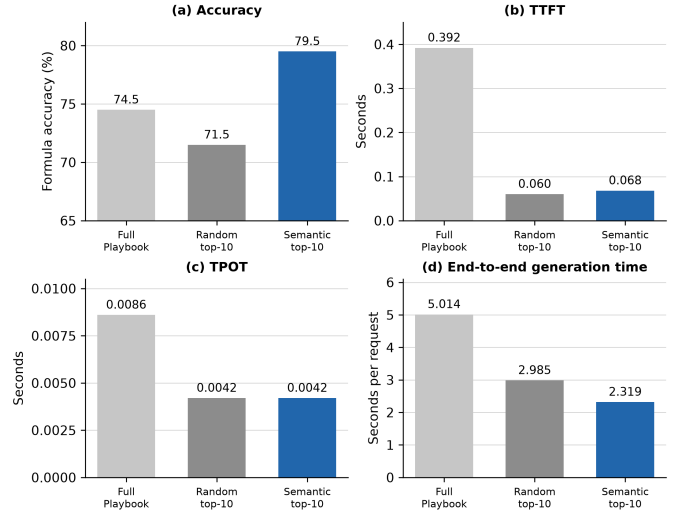


Fig. 1: Evaluation-only RAE study on Formula. All conditions use the same fixed Playbook, Qwen3-4B-Instruct-2507, and 200 test examples. Semantic top-10 retrieval is both more accurate and faster than the full-Playbook condition; random top-10 is a length-matched control.

Figure 1 shows that semantic top-10 retrieval raises accuracy from 74.5% with the full Playbook to 79.5%, while reducing mean TTFT from 0.392 to 0.068 seconds and mean end-to-end generation time from 5.014 to 2.319 seconds. The random top-10 control retains the shorter context but reaches only 71.5% accuracy. Thus, the gain is not explained by truncating the Playbook alone: the quality of query–rule matching is essential. Random retrieval lowers TTFT and TPOT almost as much as semantic retrieval, yet it removes useful guidance and performs worse than the full-Playbook condition. Conversely, passing

the entire Playbook preserves all potentially useful rules but increases prefill cost and introduces distractors. These results directly motivate Retrieval-Augmented Execution detailed in §V-B.

E. Systematic and Recurring Agent Failures

An accumulated Playbook gives ACE persistent knowledge that can be reused across tasks, but it does not guarantee that the agent’s underlying weaknesses have actually been resolved. When the base agent fails a task for a systematic reason, the same weakness can resurface on later, superficially different tasks. We therefore ask whether such failures are isolated accidents or recurring, related events.

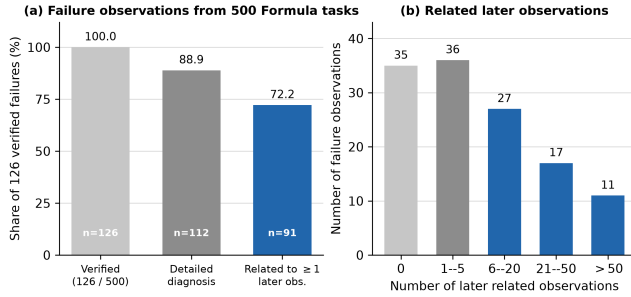


Fig. 2: Recurring failure observations in the 500-task offline Formula run, which produces 126 Ground-Truth-verified failures. Panel (a) summarizes these verified failures, and panel (b) counts, for each failure observation, how many later failure observations are semantically related to it.

The finding is that a substantial fraction of tasks produces a verified failure, and that these failures are often related across distinct tasks. As shown in Figure 2, in the 500-task offline Formula run, 126 tasks produce a Ground-Truth-verified failure (25.2% of tasks). Of these failure observations, 91 (72.2%) are related to at least one later observation, while 55 are related to at least six later observations. This pattern indicates that some underlying weaknesses can recur even when their surface questions differ.

The implication is that these recurring weaknesses should be exposed proactively, rather than only when a future user query happens to encounter them. A static Playbook cannot distinguish a robust rule from one that merely appears plausible until it is tested against a boundary case or a conflicting instruction, so a weakness that once caused a failure can remain latent in the accumulated context and resurface on later tasks until it is explicitly revised or removed. This finding directly motivates the adversarial agent and failure-memory grounding in our self-improvement loop detailed in §V-B.

V. METHODOLOGY

This section presents SCALE, the framework at the technical core of DeepEdu. Building on the empirical observations of §IV, it comprises two synergistic components that jointly address the computational and semantic challenges of deploying agentic LLMs in resource-constrained educational environments; Figure 3 gives the end-to-end picture. §V-A introduces *Similarity Chunk Rolling* (SCR), the long-context inference engine that amortizes selective sparse attention (Definition 1) from per-sub-chunk to per-cluster granularity, while §V-B introduces the self-improving agentic layer that instantiates the ACE paradigm (Definition 2) to continuously curate a localized educational playbook without expensive weight updates. The two are orthogonal yet complementary: the engine keeps the long, playbook-conditioned contexts curated by the agentic layer tractable under strict hardware budgets, while the agentic layer ensures that efficient inference stays grounded in locally accurate knowledge.

A. Efficient Long-Context Inference Engine

This subsection presents Similarity Chunk Rolling (SCR), a mechanism that reorganizes the selection-then-attention step of Equations (3)–(4) so that it runs *once per cluster* of similar sub-chunks rather than once per sub-chunk (§III-B). SCR dynamically groups consecutive sub-chunks with high mutual similarity into clusters, invokes the selection function $\mathcal{S}$ a single time per cluster, and computes the sparse attention of Equation (3) over the entire cluster in one parallel forward pass. Figure 4 illustrates the overall procedure.

1) *Cluster representation and similarity measure*: SCR represents each cluster through the sub-chunk fingerprints $\mathbf{v}_i$ of Equation (6). To make the similarity checks in the clustering inner loop cheap, we pre-normalize each fingerprint, $\tilde{\mathbf{v}}_i = \mathbf{v}_i / \|\mathbf{v}_i\|_2$, so that cosine similarity reduces to a single inner product $\cos(\mathbf{v}_i, \mathbf{v}_j) = \tilde{\mathbf{v}}_i^\top \tilde{\mathbf{v}}_j$, implementable as one fused dot-product kernel with no repeated L2-norm computation.

2) *Anchor-based cluster formation*: A naive approach to grouping similar consecutive sub-chunks would compare each sub-chunk pairwise with its immediate predecessor: extend the current cluster whenever $\cos(\mathbf{v}_i, \mathbf{v}_{i+1}) \geq \theta$. However, this sliding-pair comparison is vulnerable to a phenomenon we call drift: small incremental changes between adjacent sub-chunks may individually pass the threshold, yet accumulate across the cluster, causing the final sub-chunk to diverge substantially from

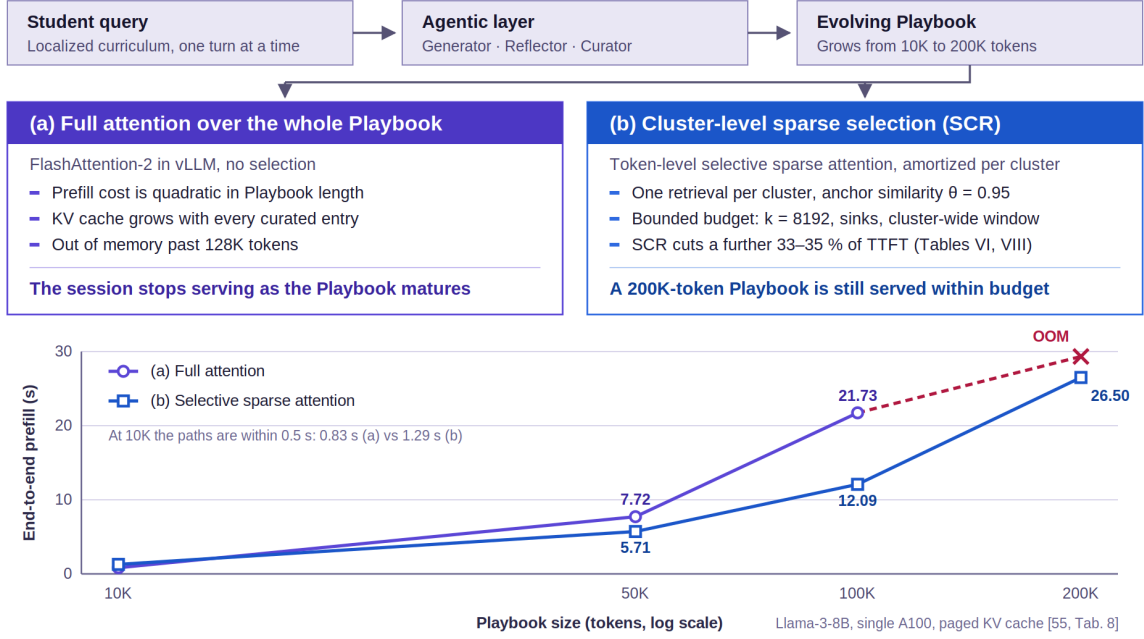


Fig. 3: SCALE pipeline comparison. As the agentic layer continuously curates an evolving playbook (10K $\rightarrow$ 200K tokens), the choice of inference engine determines deployment feasibility. Without the long-context engine, prefill latency grows quadratically and accuracy degrades on noisy long contexts, eventually triggering out-of-memory failures. SCALE’s cluster-level sparse selection maintains low latency and improves accuracy as the playbook accumulates richer localized knowledge. Latency and accuracy numbers are illustrative; concrete benchmarks are reported in Section VI.

the cluster’s starting point. The resulting cluster representation would no longer faithfully approximate any of its members.

To prevent this drift, we adopt an anchor-based procedure: the first sub-chunk $\mathbf{X}_a$ of a cluster is fixed as the anchor, and each subsequent $\mathbf{X}_j$ is admitted only while it stays similar to the *anchor*—not the drifting predecessor—and the cluster stays within a per-call memory cap $L_{\max}$ (typically $L_{\max} = 4096 = 8L$):

$$\cos(\mathbf{v}_a, \mathbf{v}_j) \geq \theta \quad \text{and} \quad |\mathcal{C}_a| \cdot L < L_{\max}. \quad (8)$$

The cluster closes at the first $\mathbf{X}_j$ violating either condition, which then becomes the next anchor—so every member is similar to a single fixed reference rather than a drifting predecessor, and no attention call exceeds $L_{\max}$ tokens. Algorithm 1 formalizes the full clustering procedure, which partitions the M sub-chunks of an outer chunk into a set of disjoint clusters $\{\mathcal{C}_1, \mathcal{C}_2, \dots\}$ in a single linear pass.

The output partition $\mathcal{P}$ contains $|\mathcal{P}| \leq M$ clusters; in the regime characterized by §IV-B—where the majority of clustering decisions yield maximum-size clusters at $\theta = 0.95$ —the reduction in retrieval invocations approaches the upper bound $|\mathcal{P}| \approx M/(L_{\max}/L)$.

Algorithm 1 Anchor-based clustering of sub-chunks.

Require: Sub-chunk fingerprints $\{\tilde{\mathbf{v}}_1, \dots, \tilde{\mathbf{v}}_M\}$; threshold θ ; max cluster size $L_{\max}$

Ensure: Clusters $\{\mathcal{C}_1, \mathcal{C}_2, \dots\}$

```

1:  $\mathcal{P} \leftarrow \emptyset, a \leftarrow 1$ 
2: while  $a \leq M$  do
3:    $\mathcal{C}_a \leftarrow \{\mathbf{X}_a\}, j \leftarrow a + 1$ 
4:   while  $j \leq M$  and  $\tilde{\mathbf{v}}_a^\top \tilde{\mathbf{v}}_j \geq \theta$  and  $|\mathcal{C}_a| \cdot L < L_{\max}$  do
5:      $\mathcal{C}_a \leftarrow \mathcal{C}_a \cup \{\mathbf{X}_j\}$ 
6:      $j \leftarrow j + 1$ 
7:   end while
8:    $\mathcal{P} \leftarrow \mathcal{P} \cup \{\mathcal{C}_a\}$ 
9:    $a \leftarrow j$ 
10: end while
11: return  $\mathcal{P}$ 

```

a) Cost model. For an outer chunk of M sub-chunks against a KV Cache of length N , the per-sub-chunk baseline invokes the selection function $\mathcal{S}$ once per sub-chunk, incurring $\mathcal{O}(MNd)$ scoring cost; SCR invokes it once per cluster, reducing this to $\mathcal{O}(|\mathcal{P}|Nd)$. At the default operating point ($\theta = 0.95$, $L_{\max} = 8L$), where most clusters reach maximum size, $|\mathcal{P}| \approx M/8$ —an $8\times$ reduction in retrieval scoring, which we measure empirically at $7.7\times$ (Figure 5). The added

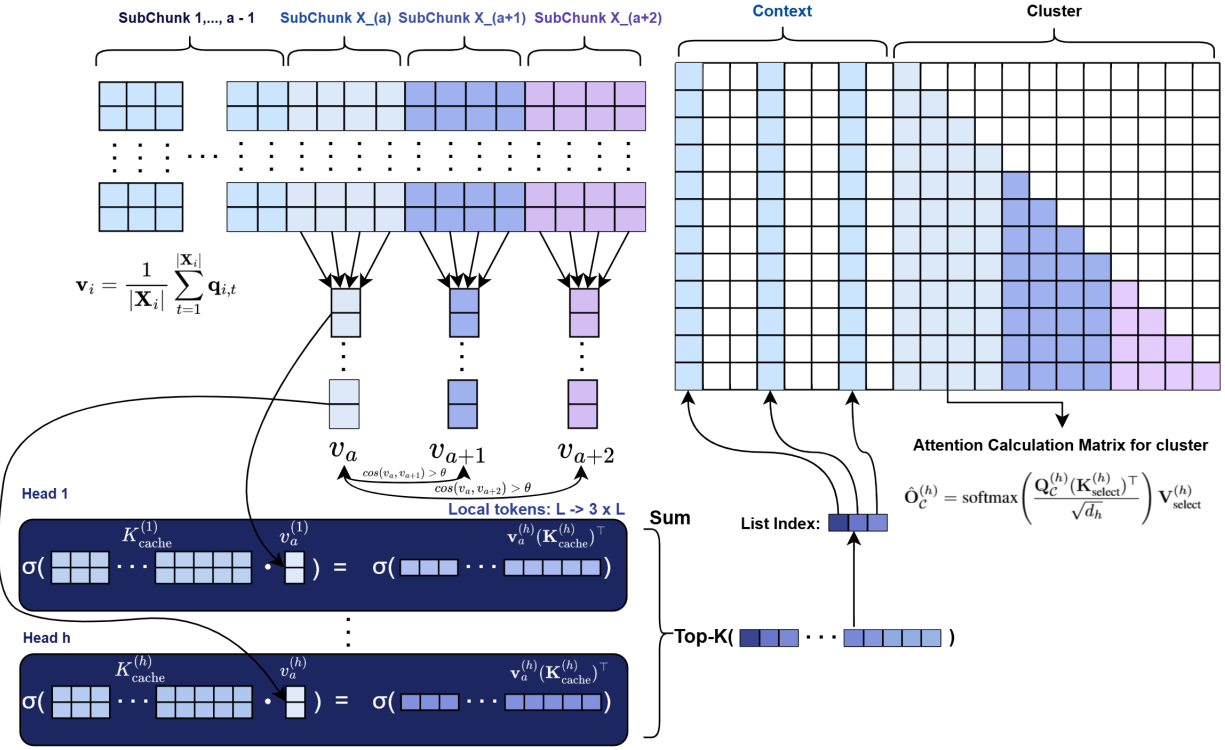


Fig. 4: Similarity Chunk Rolling (SCR) pipeline. Consecutive query sub-chunks with high mutual similarity are grouped into clusters via anchor-based formation; SCR issues a single top- k retrieval per cluster and computes sparse attention over the whole cluster in one parallel forward pass, amortizing selection from per-sub-chunk to per-cluster granularity.

bookkeeping—mean-pooling M fingerprints and one linear clustering pass (Algorithm 1)—is $\mathcal{O}(Md)$ and negligible against a single retrieval’s $\mathcal{O}(Nd)$ since $M \ll N$. The sparse-attention FLOPs are unchanged: SCR attends to the same per-query budget as the baseline but batches an entire cluster into one kernel launch instead of $|\mathcal{C}_a|$ separate calls. Because the retrieval term dominates and scales with N , the speedup grows with context length.

3) *Cluster-level retrieval and parallel attention:* For each cluster $\mathcal{C}_a$ formed by Algorithm 1, we perform a *single* selection-then-attention step over the entire cluster. Rather than aggregating its members, we reuse the cluster’s *anchor fingerprint* v_a —the query representation of the first sub-chunk X_a , against which every other member was admitted under Equation (8)—directly as the cluster-level query. The selection function $\mathcal{S}$ from Equation (4) is thus invoked once, with v_a acting as a single-row query:

$$\mathcal{I}_c = \mathcal{S}(v_a, \mathbf{K}_{\text{cache}}), \quad (9)$$

producing a single index set $\mathcal{I}_c \subseteq \{1, \dots, N\}$ with $|\mathcal{I}_c| = k$ that is shared by all members of the cluster. This single

retrieval replaces the $|\mathcal{C}|$ independent retrievals that the per-sub-chunk baseline would perform. Using the anchor rather than a mean is deliberate: the admission criterion Equation (8) guarantees $\cos(v_a, v_j) \geq \theta$ for every member $X_j \in \mathcal{C}_a$, so all members share nearly collinear query representations, and the token ranking induced by v_a closely matches the ranking each member would produce on its own. The same threshold θ that forms the cluster therefore also controls the retrieval approximation error—a fidelity we quantify empirically in §IV-C.

a) *Selection function instantiation:* We instantiate $\mathcal{S}$ using TokenSelect’s head soft-voting strategy [55]: per-head dot-product scores $v_a^{(h)} (\mathbf{K}_{\text{cache}}^{(h)})^\top$ are normalized through softmax independently, then summed across heads to produce a single criticality score per cached token, from which the top- k indices are selected. The per-head softmax normalization ensures that no single head with disproportionately large attention logits dominates the selection.

b) *Sparse attention over the cluster:* Once $\mathcal{I}_c$ is obtained, sparse attention is computed over the entire cluster in a single forward pass. We stack the cluster’s per-sub-chunk query blocks

Algorithm 2 Similarity Chunk Rolling for one outer chunk.

Require: Outer chunk queries $\mathbf{Q} \in \mathbb{R}^{\ell \times d}$; KV Cache $(\mathbf{K}_{\text{cache}}, \mathbf{V}_{\text{cache}})$; threshold θ ; sub-chunk size L ; max cluster size L_{max} ; budgets $k_{\text{base}}, n_{\text{local}}^{\text{base}}, n_{\text{init}}$

- 1: Split $\mathbf{Q}$ into $M = \lceil \ell/L \rceil$ sub-chunks $\{\mathbf{X}_1, \dots, \mathbf{X}_M\}$
- 2: Compute normalized fingerprints $\{\tilde{\mathbf{v}}_1, \dots, \tilde{\mathbf{v}}_M\}$ via Eq. (6) and L2-normalization
- 3: $\mathcal{P} \leftarrow \text{ANCHORCLUSTERING}(\{\tilde{\mathbf{v}}_i\}, \theta, L_{\text{max}})$ $\triangleright$ Algorithm 1
- 4: **for all** cluster $\mathcal{C} \in \mathcal{P}$ **do**
- 5: $\mathbf{v}_a \leftarrow$ anchor fingerprint of $\mathcal{C}$ (first sub-chunk)
- 6: $\mathcal{I}_{\mathcal{C}} \leftarrow \mathcal{S}(\mathbf{v}_a, \mathbf{K}_{\text{cache}}, k_{\text{base}})$ $\triangleright$ Eq. (9)
- 7: Augment $\mathcal{I}_{\mathcal{C}}$ with attention sinks and a cluster-spanning local window
- 8: Compute cluster attention $\hat{\mathbf{O}}_{\mathcal{C}}$ via Eq. (10) with causal mask
- 9: Write $\hat{\mathbf{O}}_{\mathcal{C}}$ to output positions of $\mathcal{C}$ in $\mathbf{O}$
- 10: **end for**
- 11: **return** $\mathbf{O}$

into $\mathbf{Q}_{\mathcal{C}} = [\mathbf{Q}_{j_1}; \dots; \mathbf{Q}_{j_{|\mathcal{C}|}}] \in \mathbb{R}^{n_{\mathcal{C}} \times d}$, where $[\cdot; \cdot]$ is row-wise concatenation and the row count $n_{\mathcal{C}} = \sum_{j \in \mathcal{C}} |\mathbf{X}_j| \leq |\mathcal{C}|L$ (equality unless $\mathcal{C}$ holds the final, shorter sub-chunk). The sparse attention output, per Equation (3), is then computed as:

$$\hat{\mathbf{O}}_{\mathcal{C}}^{(h)} = \text{softmax} \left(\frac{\mathbf{Q}_{\mathcal{C}}^{(h)} (\mathbf{K}_{\text{select}}^{(h)})^{\top}}{\sqrt{d_h}} + \mathbf{M}_{\mathcal{C}} \right) \mathbf{V}_{\text{select}}^{(h)}, \quad (10)$$

where $\mathbf{K}_{\text{select}}^{(h)}, \mathbf{V}_{\text{select}}^{(h)}$ gather three groups of KV Cache entries: the shared top- k set $\mathcal{I}_{\mathcal{C}}$ from Equation (9), retrieved once from the anchor and reused by every query in the cluster; the attention sinks (the first n_{init} tokens), and a local window. Crucially, this local window is not fixed at $n_{\text{local}}^{\text{base}}$: it is expanded to span the entire current cluster, so that every query attends to all preceding tokens within its own cluster. The anchor-shared retrieval therefore approximates only the *distant* global context, whereas all local, intra-cluster attention is computed *exactly*. After de-duplication, these three groups form $k_{\text{eff}} \leq k + n_{\text{init}} + n_{\text{local}}$ distinct entries—where n_{local} is the cluster-spanning local-window size—so that $\mathbf{K}_{\text{select}}^{(h)}, \mathbf{V}_{\text{select}}^{(h)} \in \mathbb{R}^{k_{\text{eff}} \times d_h}$ and $\hat{\mathbf{O}}_{\mathcal{C}}^{(h)} \in \mathbb{R}^{n_{\mathcal{C}} \times d_h}$; this generalizes Equation (3), whose selected set had exactly k entries. The causal mask $\mathbf{M}_{\mathcal{C}} \in \mathbb{R}^{n_{\mathcal{C}} \times k_{\text{eff}}}$ sets entry (r, s) to 0 when selected key s lies at or before the position of query row r , and to $-\infty$ otherwise, so each query attends only to keys preceding it. The whole cluster is processed in a single invocation of the FlashInfer paged attention kernel [69]. Algorithm 2 summarizes the resulting per-outer-chunk procedure end to end.

B. Self-Improving Agentic Context Engineering

This subsection presents the second component of SCALE: an agentic layer that addresses the *semantic* bottleneck of deploying LLMs in region-specific educational settings (§II, §IV-E). Rather than baking localized knowledge into model weights through expensive fine-tuning, our agentic layer continuously curates an evolving *playbook* of curriculum-specific heuristics, common student mistakes, and correct teaching trajectories. The playbook is the runtime knowledge substrate that grounds each tutoring response, and it is incrementally refined through continual interaction with learners.

We structure the agentic layer around three specialized roles—Generator, Reflector, and Curator—following the design of Agentic Context Engineering (ACE) [65], and extend it with domain-specific adaptations for educational tutoring.

1) *Agentic Architecture Overview*: Following the ACE paradigm of Definition 2, our agentic layer keeps the backbone parameters ϕ fixed and adapts the playbook $\mathcal{P}_t$ through the validated update $\mathcal{P}_{t+1} = \text{Apply}(\mathcal{P}_t, \Delta_t)$ (Equation (5); the validated Apply is made concrete in Equation (17)), where the edit set Δ_t is distilled from episode t by the Curator (§V-B4) and the feedback y_t is a reference answer, an automated grader result, or a teacher correction. Concretely, a playbook is a set of itemized entries $b_j = (\text{id}_j, \text{section}_j, \text{text}_j, h_j, a_j)$, where h_j and a_j count helpful and harmful uses, respectively.

Each episode follows a division of labor inherited from ACE [65]: the Generator G_{ϕ} produces a response and reasoning trajectory, the Reflector R_{ϕ} diagnoses that trajectory against the available feedback, and the Curator C_{ϕ} converts the resulting insight into small, structured edits. Each role is formalized where it is described, in §V-B2–§V-B4. All roles may use the same locally hosted backbone model; their different prompts and output schemas, rather than access to a stronger teacher model, create the specialization.

Our contribution extends this base loop with three complementary mechanisms: (i) Retrieval-Augmented Execution (RAE), which selects relevant rules before generation; (ii) a Failure Memory Bank (FMB), which supplies analogous past errors to the Reflector; and (iii) a lightweight adversarial curriculum, which periodically creates one playbook-conditioned stress test. These mechanisms operate at distinct points in the loop and are ablated independently in our experiments.

Algorithm 3 summarizes the complete update loop. It supports both offline adaptation over a curated corpus and online adaptation after a tutoring interaction. In either setting,

Algorithm 3 Self-improving agentic context engineering.

Require: Initial playbook $\mathcal{P}_0$; interaction stream $\mathcal{D}$; optional FMB $\mathcal{F}$; adversarial frequency f_{adv}

Ensure: Updated playbook $\mathcal{P}$ and failure memory $\mathcal{F}$

```
1: for  $(q_t, c_t, y_t) \in \mathcal{D}$  do
2:    $\mathcal{P}^{(t)} \leftarrow \text{RetrieveRules}(\mathcal{P}, q_t, c_t)$   $\triangleright$  optional RAE
3:    $(\rho_t, \hat{y}_t, U_t) \leftarrow G_\phi(q_t, c_t, \mathcal{P}^{(t)})$ 
4:    $A_t \leftarrow \text{RetrieveFailures}(\mathcal{F}, q_t)$   $\triangleright$  optional FMB
5:    $(r_t, \Gamma_t) \leftarrow R_\phi(q_t, \rho_t, \hat{y}_t, y_t, U_t, A_t)$ 
6:   Update helpful/harmful counters of entries in  $U_t$  using  $\Gamma_t$ 
7:   if  $\hat{y}_t \neq y_t$  and FMB is enabled then
8:      $\mathcal{F} \leftarrow \mathcal{F} \cup \text{Failure}(q_t, \hat{y}_t, y_t, r_t)$ 
9:   end if
10:   $\Delta_t \leftarrow C_\phi(\mathcal{P}, r_t, \Gamma_t)$ 
11:   $\mathcal{P} \leftarrow \text{Apply}(\mathcal{P}, \Delta_t)$ 
12:  if  $t \bmod f_{\text{adv}} = 0$  then
13:     $\tilde{z}_t \leftarrow \text{Adversarial}(\mathcal{P}, q_t, c_t, y_t)$ 
14:    Route  $\tilde{z}_t$  through the same Generator–Reflector–Curator loop if it exposes an error
15:  end if
16: end for
17: return  $\mathcal{P}, \mathcal{F}$ 
```

the playbook and optional failure memory persist across episodes, making the learned knowledge easy to audit, edit, and deploy under data-sovereignty constraints.

2) *Generator - Producing Tutoring Trajectories:* The Generator is prompted with the student query, instructional context, and a structured subset of the current playbook. Formally, it returns a tutoring trajectory ρ_t , an answer $\hat{y}_t$, and the identifiers U_t of the entries used in its reasoning,

$$(\rho_t, \hat{y}_t, U_t) = G_\phi(q_t, c_t, \mathcal{P}_t), \quad (11)$$

where $\mathcal{P}_t$ is the playbook supplied to the call—the focused subset selected by RAE (Equation (19), §V-B5) when it is enabled, otherwise the full playbook. Requiring entry identifiers makes playbook use observable: the subsequent Reflector can attribute a successful or failed response to concrete advice rather than treating the context as an opaque prompt.

For tutoring, ρ_t can include a worked explanation, a Socratic hint sequence, or a tool-use trace, depending on the learning objective. The response contract separates pedagogical form from factual content: the Generator must answer the question while exposing enough intermediate reasoning for feedback to identify whether the failure arose from a misunderstood local fact, an inappropriate instructional strategy, or a missing prerequisite. This trajectory is not retained as hidden model

state; it is transient evidence for reflection and curation.

3) *Reflector - Extracting Insights from Execution Feedback:* The Reflector receives the student query q_t ; the trajectory ρ_t , answer $\hat{y}_t$, and cited entries U_t produced by the Generator (Equation (11)); and the strongest feedback signal y_t available. During offline training this signal can be a reference answer or deterministic grader; during deployment it can be a teacher correction, tool result, or explicit student follow-up. Formally, it maps these inputs to a structured reflection r_t and a set of tags Γ_t over the cited entries,

$$(r_t, \Gamma_t) = R_\phi(q_t, \rho_t, \hat{y}_t, y_t, U_t), \quad (12)$$

where each cited entry receives a tag in $\{\text{HELPFUL}, \text{NEUTRAL}, \text{HARMFUL}\}$ and the reflection itself is

$$r_t = (e_t, z_t, a_t^*, \kappa_t), \quad (13)$$

where e_t identifies the observed error, z_t explains its root cause, a_t^* specifies a correct approach, and κ_t states a reusable key insight.

This explicit diagnosis is important for localized tutoring. For example, an answer can be linguistically fluent yet fail because it uses an incorrect regional historical fact, conflates two curriculum definitions, or gives a pedagogically unsuitable shortcut. The Reflector converts such instance-level feedback into an actionable explanation that can be reused without exposing the original student’s private record. Multiple reflection rounds may be used when a first diagnosis is incomplete; the final reflection is passed to the Curator. Because a reflection is later converted into persistent context, the Reflector is also the critical point at which hallucinated diagnosis can enter the long-horizon learning loop. We therefore preserve the evidence, cited bullet identifiers, and the resulting delta as auditable artifacts, rather than treating a reflection as an untraceable free-form summary.

a) *Failure Memory Bank:* To support analogical reflection, SCALE maintains a Failure Memory Bank (FMB), separate from the rule-oriented playbook. A memory record pairs the failed question, prediction, and reference outcome with the error e_i , root cause z_i , and key insight κ_i distilled by the reflection (Equation (13)):

$$f_i = (q_i, \hat{y}_i, y_i, e_i, z_i, \kappa_i, \mathbf{e}_i), \quad (14)$$

where $\mathbf{e}_i = \text{norm}(E(q_i))$ is an in-memory embedding used only for retrieval. A record is added after reflection when

the observed answer disagrees with the available reference outcome, so the record contains distilled diagnosis rather than an unprocessed trajectory. For a new query q , normalized embedding vectors retrieve the K closest prior failures:

$$\mathcal{N}_K(q) = \text{TopK}_{f_i \in \mathcal{F}} \frac{E(q)^\top E(q_i)}{\|E(q)\|_2 \|E(q_i)\|_2}. \quad (15)$$

The retrieved cases are rendered as additional context for the Reflector, allowing it to recognize recurring failure patterns. FMB therefore does not replace the playbook: the playbook guides execution, whereas FMB supplies episodic evidence to improve diagnosis.

4) *Curator - Incremental Playbook Updates*: The Curator turns the reflection and entry tags (r_t, Γ_t) of Equation (12) into a compact set of typed delta operations Δ_t , rather than rewriting the full prompt:

$$\Delta_t = C_\phi(\mathcal{P}_t, r_t, \Gamma_t). \quad (16)$$

The operational path primarily uses an ADD request over a named playbook section; the surrounding schema also accommodates targeted update, merge, and delete operations as the playbook maintenance policy evolves. The JSON operation schema is validated before application; malformed output yields an empty delta and leaves the existing playbook unchanged. This safety guard is the concrete definition of the Apply operator of Equation (5):

$$\text{Apply}(\mathcal{P}_t, \Delta_t) = \begin{cases} \Phi(\mathcal{P}_t, \Delta_t), & \text{if } \Delta_t \text{ is valid,} \\ \mathcal{P}_t, & \text{otherwise,} \end{cases} \quad (17)$$

where Φ is deterministic, non-LLM application logic. Periodic merging of redundant entries (the “refine” step of the grow-and-refine policy) is applied separately and is described below.

Entries are organized by reusable strategies, common mistakes, worked templates, and troubleshooting guidance. The Curator follows a grow-and-refine policy: it appends a new entry when the reflection exposes a genuinely absent rule; otherwise it updates usage counters or merges overlapping entries. This localized update mechanism preserves prior knowledge and prevents the context-collapse failure mode associated with repeated monolithic prompt rewrites.

5) *Incremental Updates and Context Collapse Prevention*: As the playbook grows, passing every entry to every generation is both inefficient and vulnerable to distraction from irrelevant rules. We use Retrieval-Augmented Execution (RAE) to select only the most relevant playbook bullets before invoking the

Generator. Each entry $b_j = (\text{id}_j, \text{section}_j, \text{text}_j, h_j, a_j)$ from the schema defined above contributes its bullet text text_j , which is embedded and L2-normalized. The implementation parses the structured playbook, indexes all bullet texts with FAISS inner-product search, and retains section headers when reconstructing a focused prompt. For query context $u_t = q_t \oplus c_t$, RAE computes

$$\mathbf{e}_j = \text{norm}(E(\text{text}_j)), \quad \mathbf{e}_q = \text{norm}(E(u_t)), \quad (18)$$

$$I_K = \text{TopK}_j \mathbf{e}_q^\top \mathbf{e}_j, \quad (19)$$

and supplies the focused playbook $\mathcal{P}^{(t)} = \{b_j \mid j \in I_K\}$ to the Generator. Section headers are retained so that retrieved bullets preserve their original organizational context. The index is rebuilt after each accepted Curator update; if the index is unavailable or the playbook contains at most K entries, the system safely falls back to the full playbook.

RAE is deliberately placed on the execution path, rather than used only for post-hoc analysis: the same retrieval call conditions initial generation, post-reflection regeneration, post-Curator generation, validation, and adversarial execution. It therefore reduces both prompt noise and inference cost whenever the playbook becomes long, while entry identifiers still allow the Reflector to assign credit or blame to the rules actually used.

RAE and FMB share the same embedding encoder when both are enabled, avoiding a duplicate model copy on memory-constrained hardware. RAE retrieves rules for the Generator, while FMB retrieves past failures for the Reflector; this separation prevents a past erroneous trajectory from being directly treated as an execution instruction.

To bound long-run context growth, SCALE periodically compares entry embeddings and merges semantically redundant entries when their cosine similarity exceeds a configurable threshold. Pruning or deduplication is performed only through validated, localized operations. Consequently, an update to one educational rule neither discards unrelated rules nor requires re-generating the entire playbook.

6) *Adversarial Curriculum*: We use a lightweight adversarial curriculum during adaptation to expose playbook weaknesses not covered by routine interactions. For the Formula and FiNER experiments it is invoked when $t \bmod f_{\text{adv}} = 0$; for AppWorld it runs after the original task trajectory has completed. A single adversarial LLM call, conditioned on the current playbook and a recent training sample, returns exactly one plausible

challenge

$$\tilde{z}_t = (\tilde{q}_t, \tilde{c}_t, \tilde{y}_t, \tilde{a}_t, \tilde{v}_t), \quad (20)$$

containing a question, minimal context, target answer, attack rationale, and vulnerability hint. The standard Generator executes this challenge under the current playbook and the data processor compares its answer against the generated target. If $\hat{y}_t \neq \tilde{y}_t$, the resulting trajectory is routed through the same Generator–Reflector–Curator loop as a genuine failure (Equations (11)–(16)), and can also enter the FMB after reflection, letting a synthetically exposed weakness inform later analogical diagnosis. The adversarial component therefore creates additional training signals, but never edits the playbook directly. Since this mechanism uses one generated target without an independent verifier, we treat it as active stress testing rather than formal evidence of an error. This distinction is essential: the contribution is a low-cost curriculum generator, not a claim of adversarial verification.

VI. EXPERIMENTS AND RESULTS

We evaluate SCALE in three parts. Because the two components target different bottlenecks and call for different baselines, we first study each in isolation: the long-context inference engine (§VI-A) is measured against a state-of-the-art selective sparse attention method on long-context benchmarks, whereas the self-improving agentic layer (§VI-B) is evaluated as a controlled ablation over its mechanisms on agentic reasoning benchmarks. We then measure the two together in the configuration DeepEdu actually deploys—the agentic layer running on top of SCR (§VI-C).

A. Long-Context Inference Engine

We evaluate the long-context inference engine of SCALE (§V-A) against a strong selective sparse attention baseline, TokenSelect [55], on two established long-context benchmarks. We aim to answer three research questions:

- **RQ1:** Does Similarity Chunk Rolling (SCR) reduce prefill latency compared to per-sub-chunk selective sparse attention?
- **RQ2:** Does the cluster-level retrieval paradigm preserve or improve task accuracy compared to per-sub-chunk retrieval?
- **RQ3:** How sensitive is SCR to its hyperparameters, specifically the similarity threshold θ and the maximum cluster size $L_{\max}$?

1) Experimental Setup:

a) *Hardware and software:* All experiments are conducted on a single NVIDIA H100 GPU with 80 GB HBM3. We implement SCR on the SGLang [70] serving framework with the FlashInfer [69] attention backend, in full bfloat16 precision. The backbone model is Qwen2-7B-Instruct [16], configured with a maximum context length of 1,048,576 tokens, RoPE base 10^6 , outer chunk size $L_{\text{outer}} = 8192$, and inner sub-chunk size $L = 512$.

b) *Baseline:* Our primary baseline is **TokenSelect** [55], the current state-of-the-art query-aware token-level selective sparse attention method. TokenSelect invokes its selection function $\mathcal{S}$ independently per sub-chunk during prefill, providing a direct comparison point against SCR’s cluster-level amortization. We use the same selection function (head soft-voting), retrieval budget ($k_{\text{base}} = 8192$), base local window ($n_{\text{local}}^{\text{base}} = 512$), and attention sinks ($n_{\text{init}} = 128$) for both methods. By construction, however, SCR’s local window expands to span each cluster (§V-A3), so the comparison reflects the full cluster-level mechanism: amortized retrieval, which drives the latency savings, together with the wider cluster-spanning local attention, which drives the accuracy gains on structured-retrieval tasks.

c) *SCR configurations:* We evaluate SCR across the full grid of two hyperparameters: similarity threshold $\theta \in \{0.95, 0.97, 0.99\}$ and maximum cluster size $L_{\max} \in \{1024, 2048, 4096\}$ (i.e., 2, 4, and 8 sub-chunks per cluster), yielding nine configurations.

d) *Benchmarks:* We use two standard long-context benchmarks:

- **InfiniteBench** [67]: six tasks spanning code debugging (Code.D), structured retrieval (key–value retrieval R.KV, number string R.Num, passkey R.PK), narrative dialogue QA (En.Dia), and math find (Math.F); we use these abbreviations, following [55], throughout. Each task is evaluated on its full test set with native long-context inputs.
- **RULER** [71]: eight needle-in-a-haystack (NIAH) tasks—single-, multi-key, multi-value, and multi-query variants—at 100 samples per task across six context lengths from 4096 to 131,072 tokens, isolating retrieval accuracy as context length grows.

We report average accuracy across tasks and average time-to-first-token (TTFT) latency in seconds, with TTFT measured from the start of prefill to the emission of the first decoded token.

TABLE VI: Per-task accuracy (%) on representative InfiniteBench tasks (Qwen2-7B-Instruct). R.Num and R.PK are saturated at 100% and omitted here; the full six-task, nine-configuration grid is in Table XII. Best per column in bold.

Method	Code.D	R.KV	En.Dia	Math.F
Qwen2-7B-Instruct [16]	28.2	19.0	8.5	19.7
NTK [34]	24.9	59.8	7.5	27.7
SelfExtend [72]	8.1	0.0	5.0	2.3
StreamingLLM [48]	27.9	2.4	3.5	19.4
InfLLM [51]	27.4	5.4	7.5	24.0
TokenSelect [55]	26.7	91.0	10.0	24.0
$\theta=0.95, L_{\max}=1024$	27.6	94.0	10.0	23.7
$\theta=0.95, L_{\max}=2048$	26.1	96.2	10.0	23.4
$\theta=0.95, L_{\max}=4096$	24.9	98.0	6.0	22.6
$\theta=0.99, L_{\max}=1024$	28.8	94.0	11.5	23.1

2) Accuracy and Latency on InfiniteBench:

a) *Accuracy*: Table VI reports per-task accuracy on representative InfiniteBench tasks (the full six-task grid is in Appendix A). SCR matches or exceeds the TokenSelect baseline on nearly all tasks. Most notably, on R.KV—a task with long structured contexts that historically stresses sparse attention—SCR with $L_{\max}=4096$ reaches 98.0% versus the baseline’s 91.0%, a **7-point absolute improvement**, and this accuracy grows monotonically with cluster size ($94.0 \rightarrow 96.2 \rightarrow 98.0\%$ as $L_{\max}$ increases from 1024 to 4096). We attribute this gain not to amortization—which affects only latency—but to the wider effective local window that cluster-level processing yields as an intrinsic byproduct, preserving more local context relevant to structured retrieval. Crucially, SCR obtains this wider window *while* amortizing retrieval; a baseline could match it only by widening its own per-sub-chunk attention, at the cost of the latency advantage. SCR also edges out the baseline on Code.D (28.8% vs. 26.7%) and En.Dia (11.5% vs. 10.0%), while R.Num and R.PK are saturated at 100% for both TokenSelect and every SCR configuration. Against the other long-context baselines the margin on structured retrieval is far larger: the strongest competing sparse method on R.KV (NTK) reaches only 59.8%, and memory-based methods such as InfLLM fall below 6%, underscoring how much cluster-level selection preserves.

b) *Latency*: Table VII reports TTFT latency per task. SCR delivers substantial latency reductions over the baseline on all tasks. Averaging across tasks, SCR with $\theta=0.95$ and $L_{\max}=4096$ achieves roughly a 35% **latency reduction**: from 13.2s to 8.3s on R.KV, from 8.8s to 5.7s on Code.D, and from 8.7s to 5.7s on Math.F.

TABLE VII: Per-task TTFT latency (s) on representative InfiniteBench tasks. SCR delivers roughly a 35% latency reduction over the TokenSelect baseline (top- $k=8192$), with the largest gains on long-context retrieval tasks. The complete nine-configuration grid over all six tasks appears in Table XIII. Best (lowest) per column in bold.

Method	Code.D	R.KV	En.Dia	Math.F
TokenSelect	8.8	13.2	7.7	8.7
$\theta=0.95, L_{\max}=1024$	6.7	9.9	6.0	6.6
$\theta=0.95, L_{\max}=2048$	5.9	8.7	5.3	5.9
$\theta=0.95, L_{\max}=4096$	5.7	8.3	5.1	5.7
$\theta=0.99, L_{\max}=1024$	7.3	9.9	6.2	6.7

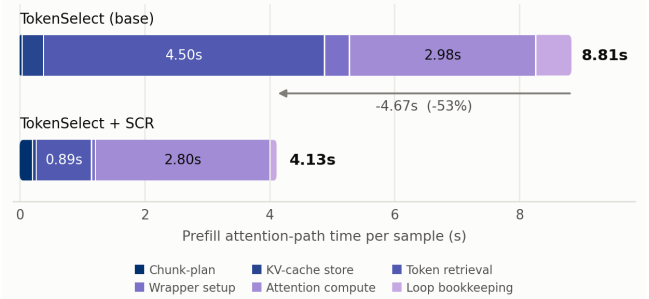


Fig. 5: Per-sample prefill time by stage on InfiniteBench R.KV (Qwen2-7B-Instruct; separate instrumented run, $n=30$). Overall TTFT falls 13.1s $\rightarrow$ 8.3s (−37%); almost the entire reduction comes from the token-retrieval stage, while attention compute is nearly unchanged.

c) *Where the speedup comes from*: To see *why* SCR is faster, we instrument the prefill pass and attribute wall-clock time to each stage (Figure 5, R.KV, $n=30$). Almost the entire gain comes from a single stage—token retrieval—which falls from 4.5 to 0.9s per sample (77% of the total saving), because SCR issues $7.7\times$ fewer retrieval calls ($8,959 \rightarrow 1,165$), close to the analytic $8\times$ bound (§V-A2). Attention compute barely changes (−4%) even though SCR processes $5.5\times$ fewer KV tokens: the win is in how *often* the model selects tokens, not in the attention FLOPs. As retrieval cost grows with context length, this saving only widens for the longer contexts that motivate SCR.

3) Scaling with Context Length on RULER:

a) *Accuracy*: Table VIII reports average accuracy across the eight RULER tasks at six context lengths. SCR variants match or slightly exceed the TokenSelect baseline at every context length, confirming that cluster-level amortization does not compromise retrieval fidelity. At the longest context of 128K tokens, the best SCR configuration ($L_{\max}=4096$) achieves 65.28% accuracy versus the baseline’s 64.78%, a

TABLE VIII: Average accuracy (%) on RULER across eight needle-in-a-haystack tasks at varying context lengths. SCR variants preserve or slightly improve over the TokenSelect baseline at every context length.

Method	4K	8K	16K	32K	64K	128K
TokenSelect	96.53	97.72	95.16	78.69	69.09	64.78
$\theta = 0.95, L_{\max} = 1024$	96.75	97.59	95.28	78.53	68.66	64.88
$\theta = 0.95, L_{\max} = 4096$	96.69	97.81	95.97	77.81	69.72	65.28

TABLE IX: Average TTFT latency (s) on RULER at varying context lengths. SCR achieves substantial latency reduction at long contexts, with the gap widening as context length grows.

Method	4K	8K	16K	32K	64K	128K
TokenSelect	0.52	0.65	1.94	3.17	5.27	9.89
$\theta = 0.95, L_{\max} = 1024$	0.53	0.65	1.88	2.94	4.57	8.16
$\theta = 0.95, L_{\max} = 4096$	0.52	0.62	1.75	2.64	3.90	6.64

small but consistent improvement.

b) Latency: Table IX reports average TTFT in seconds. SCR delivers progressively larger latency savings as context length grows. At 4K–8K tokens, SCR variants and baseline are within 5% of each other (both near 0.5–0.7 s), as retrieval cost is negligible at short contexts. At 128K tokens, SCR with $L_{\max} = 4096$ achieves 6.64 s versus the baseline’s 9.89 s, a 32.9% **latency reduction**. The advantage widens super-linearly with context length—3.0% at 16K, 16.7% at 32K, 25.9% at 64K, and 32.9% at 128K—because longer prefills process more outer chunks, creating more clustering opportunities and a growing cumulative reduction in retrieval invocations. This confirms that the amortization predicted by §IV materializes in wall-clock latency on real hardware.

4) Ablation: Effect of $L_{\max}$ and Threshold θ :

a) Effect of maximum cluster size $L_{\max}$: Increasing $L_{\max}$ from 1024 to 4096 at fixed $\theta = 0.95$ monotonically reduces latency—on R.KV, TTFT drops from 9.9 s to 8.3 s—confirming that larger clusters yield more amortization. The accuracy effect is task-dependent: structured retrieval benefits from larger clusters (R.KV climbs from 94.0% to 98.0% as $L_{\max}$ grows from 1024 to 4096), whereas code and dialogue reasoning slightly favor smaller clusters (Code.D and En.Dia peak at $L_{\max} = 1024$). This asymmetry reflects the trade-off between amortization gain and selection granularity: when clusters become large, the single anchor fingerprint $\mathbf{v}_a$ must stand in for an increasingly wide span of sub-chunks, so members further from the anchor may carry retrieval needs it does not fully capture. This helps tasks with locally coherent context

but mildly hurts tasks requiring fine-grained per-sub-chunk selection. We adopt $L_{\max} = 4096$ as default for its consistent latency advantage and strong structured-retrieval accuracy, noting that practitioners may reduce $L_{\max}$ when fine-grained reasoning is paramount.

b) Effect of similarity threshold θ : Sweeping $\theta \in \{0.95, 0.97, 0.99\}$ at fixed $L_{\max}$ (full grid in Appendix A) reveals only minor accuracy variation (± 2 points) with near-identical latency, consistent with the cluster size distribution observed in §IV-B: across this range the majority of clustering decisions still reach the maximum cluster size, so the effective amortization is largely insensitive to θ . We adopt $\theta = 0.95$ as default for its slightly more aggressive amortization, with higher thresholds available when finer cluster granularity is desired.

B. Self-Improving Agentic Layer

We evaluate the self-improving agentic layer as a controlled ablation over its three added mechanisms—Retrieval-Augmented Execution (RAE), the Failure Memory Bank (FMB), and the adversarial curriculum—independently of the TokenSelect comparison. We aim to answer three research questions:

- **RQ4:** Do RAE, FMB, and the adversarial curriculum improve grounded financial reasoning on Formula and FiNER under the same offline-learning budget?
- **RQ5:** Do these mechanisms transfer to interactive ApWorld agents, including both normal and challenge trajectories?
- **RQ6:** As selective retrieval, failure memory, and self-initiated stress testing are added incrementally, how does each mechanism contribute, and where does it introduce task-specific trade-offs?

1) Experimental Setup:

a) Setup and baseline: All Generator, Reflector, and Curator roles use Qwen3-4B-Instruct-2507, served through vLLM on the same single H100 80 GB GPU as the SCR experiments. Supervision comes only from Ground Truth labels or deterministic environment outcomes—no configuration learns from unverified live student interactions—reflecting the intended educational workflow, in which the system first curates reliable knowledge from verified data and then deploys the resulting Playbook. Our baseline is the original ACE architecture [65], a Generator–Reflector–Curator loop that accumulates a structured Playbook from Ground Truth

feedback; the **Origin** row in Tables X and XI is this baseline with RAE, FMB, and the adversarial curriculum disabled. Every variant shares Origin’s backbone, supervision, seed, token budgets, and evaluation protocol, so the ablation isolates the effect of the three added mechanisms.

b) Training configuration for Formula and FiNER: We train for one epoch with at most three agentic rounds per instance, and a 2,048-token generation limit. We use seed 42, evaluate every 50 steps, save every 25 steps, and run evaluation with five workers. RAE and FMB each retrieve the top $K = 10$ entries. The adversarial curriculum is scheduled every $f_{\text{adv}} = 10$ update steps. Thus, the ablation changes only the switches (RAE, FMB, Adv); all backbone, optimization, and evaluation settings are held fixed.

c) AppWorld protocol and reported metrics: Among the ACE benchmarks, AppWorld is the closest analogue to our target setting of AI-assisted learning: it requires an agent to carry out interactive, multi-step, tool-augmented tasks in a stateful environment, recovering from its own execution errors along the way—exactly the mode in which a tutoring agent must operate on a learner’s behalf, as opposed to answering a single self-contained question. Its normal and challenge splits therefore serve as the shared testbed for both SCALE components in this setting: they let us probe whether the self-improving loop improves task success (§VI-B) and whether the SCR engine sustains low latency (§VI-C), precisely where educational deployment is hardest—sustained, error-prone, multi-turn interaction. For AppWorld we use the same Qwen3-4B-Instruct-2507 backbone and seed 42, with RAE and FMB again configured at $K = 10$. The environment’s task completion signal is treated as Ground Truth. Unlike the step-scheduled Formula/FiNER protocol, an AppWorld adversarial case is generated only after the original task trajectory finishes. This ensures that the synthetic case probes a completed, observable trajectory and does not replace or contaminate the original task evaluation. We report final test accuracy on Formula and FiNER, and Task Goal Completion (TGC) and Scenario Goal Completion (SGC) on the AppWorld normal and challenge splits.

2) Financial Reasoning: Formula and FiNER: Table X reports an incremental ablation that adds the three mechanisms on top of the Origin Playbook, which obtains 70.0 on Formula and 51.3 on FiNER (average 60.7). Adding RAE raises Formula to 77.5 by supplying the Generator with a compact set of highly relevant rules; FiNER dips to 47.6, indicating that it can

instead benefit from broader complementary context. Layering the adversarial curriculum on top yields the strongest financial-reasoning configuration, lifting Formula to 79.5 and recovering FiNER to 52.1 for the best average (65.8); boundary-case stress tests expose rule gaps that routine training misses.

Adding the Failure Memory Bank completes the full deployed configuration. It trades a little Formula accuracy (73.0) for the best FiNER score (52.9), reflecting FMB’s role in supplying analogous, Ground Truth-backed failures that stabilize cross-task behavior—an effect that matters most on the interactive AppWorld agents examined next, where the same failures recur across multi-step trajectories.

3) Interactive Agents: AppWorld: Table XI follows the same cumulative order on AppWorld, using Task Goal Completion (TGC) and Scenario Goal Completion (SGC) on the normal and challenge splits. Adding RAE lifts the Origin average from 9.3 to 10.4, improving normal SGC ($8.9 \rightarrow 12.5$) and challenge TGC ($6.7 \rightarrow 8.2$) by focusing the Generator on relevant rules. The adversarial curriculum, valuable on the static financial tasks, does not by itself transfer to interactive execution: layered on RAE it improves only challenge SGC ($0.7 \rightarrow 2.2$) while the aggregate slips to 8.9, since a synthetically exposed weakness yields lasting benefit only once it is diagnosed and retained.

Adding the Failure Memory Bank supplies exactly this retention: the full deployed configuration recovers to the best average (10.4) and the best normal TGC (22.0), confirming that verified failure recall is what makes the adversarial signal durable when an agent must recover from recurrent multi-step execution errors. Taken together, the two benchmark groups tell a consistent story: RAE focuses execution, the adversarial curriculum surfaces latent weaknesses, and the Failure Memory Bank retains their lessons, so the full configuration is the most balanced across the static and interactive tasks.

C. End-to-End Efficiency: SCR on the Agentic Layer

The two preceding subsections evaluated each component on its own bottleneck and baselines. We now measure them jointly, running the self-improving agentic layer on top of the SCR inference engine—the configuration DeepEdu actually deploys—to verify that the amortized long-context engine translates into concrete latency gains for the agent, whose every turn re-prefills a Playbook-conditioned context.

a) Setup: We measure inference efficiency for ACE on the AppWorld normal and challenge splits, using the same Playbook and Qwen3-4B-Instruct-2507 backbone on the H100

TABLE X: Incremental ablation of the agentic mechanisms on Formula and FiNER using Ground Truth labels. Mechanisms are added cumulatively on top of the Origin Playbook; all runs use Qwen3-4B-Instruct-2507, seed 42, and identical training settings. Average is the mean of the two accuracies; the final row is the full configuration DeepEdu deploys. Best per column in bold.

Method	RAE	FMB	Adv.	FiNER Acc↑	Formula Acc↑	Average↑
Origin	–	–	–	51.3	70.0	60.7
+ RAE	✓	–	–	47.6	77.5	62.6
+ Adversarial	✓	–	✓	52.1	79.5	65.8
+ FMB (DeepEdu)	✓	✓	✓	52.9	73.0	63.0

TABLE XI: Incremental ablation of the agentic mechanisms on AppWorld using Ground Truth environment outcomes. Mechanisms are added cumulatively on top of Origin; TGC and SGC are reported for the normal and challenge splits, and Average is the mean of the four reported metrics. The final row is the full configuration DeepEdu deploys. Best per column in bold.

Method	RAE	FMB	Adv.	Test-Normal		Test-Challenge		Average
				TGC↑	SGC↑	TGC↑	SGC↑	↑
Origin	–	–	–	20.8	8.9	6.7	0.7	9.3
+ RAE	✓	–	–	20.2	12.5	8.2	0.7	10.4
+ Adversarial	✓	–	✓	20.2	7.1	6.2	2.2	8.9
+ FMB (DeepEdu)	✓	✓	✓	22.0	10.7	7.4	1.4	10.4

platform of §VI-B1. The **Origin** configuration is the standard ACE implementation served through vLLM; **SCR** replaces its long-context execution path with our Similarity Chunk Rolling engine (§V-A) while keeping the ACE task interface and splits unchanged. We report time-to-first-token (TTFT), time-per-output-token (TPOT), and aggregate input/output tokens.

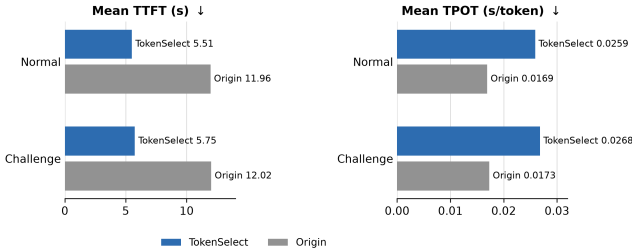


Fig. 6: Aggregate inference statistics for ACE on the AppWorld normal and challenge splits

b) *TTFT*: Figure 6 reports latency on the two AppWorld splits. SCR reduces mean TTFT from 11.96 to 5.51 seconds on Normal (53.9% reduction; $2.17\times$ faster) and from 12.02 to 5.75 seconds on Challenge (52.2% reduction; $2.09\times$ faster). This reduction is valuable for ACE because each agent turn begins by prefilling a Playbook-conditioned context.

c) *TPOT and token consumption*: SCR itself targets the prefill stage: the TTFT reductions above are its direct effect, and it leaves the decode path unchanged. Per-token decode latency (TPOT) is instead governed by the *selection cache* of the underlying TokenSelect engine [55], which caches token-

selection results across consecutive, highly similar decode queries. Because SCR is built on top of TokenSelect, the deployed engine retains this cache and keeps TPOT low at decode; TokenSelect reports that the mechanism lowers decode latency and yields up to a $2.28\times$ end-to-end speedup. The two effects are complementary—SCR amortizes selection at prefill, the selection cache amortizes it at decode—so the deployed engine stays fast in both phases (Figure 6). The deployed engine also processes fewer input tokens on both splits, consistent with selective context execution; total output-token counts differ across the evaluated runs, so they should not be interpreted as an isolated throughput comparison.

VII. CONCLUSION

We presented DeepEdu-v1, an AI-tutoring system for Vietnamese education deployable on the constrained, on-premise hardware that data-sovereignty regulations require. Its technical core is SCALE, a framework that couples two components targeting the two bottlenecks such deployments face. *Similarity Chunk Rolling* (SCR) attacks the dynamic KV-cache and prefill-latency bottleneck by amortizing selective sparse attention from per-sub-chunk to per-cluster granularity, sustaining low latency as context grows. The *self-improving agentic layer* attacks the semantic bottleneck: rather than costly weight updates, it continuously curates an evolving Playbook, using a failure-memory bank and an adversarial curriculum to proactively

expose and repair the systematic, recurring weaknesses we observed in the base agent.

Our evaluation shows that these components are effective both separately and together. On long-context benchmarks SCR preserves accuracy while cutting prefill latency, and running the agentic layer on top of SCR reduces mean time-to-first-token by roughly $2\times$ ($2.17\times$ on the AppWorld normal split and $2.09\times$ on challenge)—the configuration DeepEdu actually deploys—confirming that the two contributions compose rather than merely coexist. Because SCR builds on TokenSelect, the deployed engine also inherits its selection cache, which keeps per-token decode latency low. The self-improving layer also lifts final accuracy on complex tasks from 70.0% to 79.5% and yields the strongest per-track gains across finer-grained sub-tasks and adversarial splits, breaking the performance ceiling of the underlying framework without any fine-tuning.

One main limitation frames our future work: our experiments validate the enabling mechanisms on established long-context and agentic benchmarks, whereas evaluating the localized grounding claim directly on native, region-specific curricula—the setting that ultimately motivates DeepEdu—remains important future work toward democratizing trustworthy AI tutoring in resource-constrained regions.

APPENDIX

Tables XII and XIII report the complete results for all nine SCR configurations ($\theta \in \{0.95, 0.97, 0.99\}$, $L_{\max} \in \{1024, 2048, 4096\}$) across all six InfiniteBench tasks, including R.Num and R.PK, which are saturated at 100% accuracy for every method. The representative subsets in the main-text Tables VI and VII are drawn from these grids.

REFERENCES

- [1] H. Touvron, L. Martin, K. Stone *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint*, 2023.
- [2] J. Bai, S. Bai, Y. Chu *et al.*, “Qwen technical report,” *arXiv preprint*, 2023.
- [3] S. Yao, J. Zhao, D. Yu *et al.*, “React: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [4] S. Wang, T. Xu, H. Li *et al.*, “Large language models for education: A survey and outlook,” *IEEE Signal Processing Magazine*, vol. 42, no. 6, pp. 51–63, 2025.
- [5] Government of Vietnam, “Decree no. 53/2022/nd-cp detailing a number of articles of the law on cybersecurity,” <https://luatvietnam.vn/an-ninh-quoc-gia/nghi-dinh-53-2022-nd-cp-228170-d1.html>, 2022, introduces cybersecurity and data governance requirements in Vietnam, including conditions for data localization and local data storage obligations for certain domestic and foreign digital service providers.
- [6] S. Cahyawijaya, H. Lovenia, and P. Fung, “LLMs are few-shot in-context low-resource language learners,” in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2024.
- [7] VnExpress, “Teachers and students enjoy experimenting with chatgpt,” <https://vnexpress.net/giao-vien-hoc-sinh-thich-thu-trai-nghiem-chatgpt-4565643.html>, 2023, highlights the increasing adoption of AI assistants by Vietnamese students and teachers, alongside concerns over factual inconsistencies.
- [8] H. Trang, “The dangers of using chatgpt to distort vietnamese history,” <https://www.doanhnhantrevietnam.vn/nguy-hai-khi-ung-dung-chatgpt-xuyen-tac-ve-lich-su-viet-nam-d18556.html>, 2023, details severe historical hallucinations by ChatGPT, such as conflating Quang Trung and Nguyen Hue as different kings or fabricating facts about national leaders.
- [9] J. Lin, J. Tang, H. Tang *et al.*, “Awq: Activation-aware weight quantization for on-device llm compression and acceleration,” in *Proceedings of Machine Learning and Systems*, 2024, pp. 87–100.
- [10] E. Frantar, S. Ashkboos, T. Hoeffer, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” in *The Eleventh International Conference on Learning Representations*, 2023.
- [11] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, “Flashattention: Fast and memory-efficient exact attention with io-awareness,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022, pp. 16 344–16 359.
- [12] W. Kwon, Z. Li, S. Zhuang *et al.*, “Efficient memory management for large language model serving with pagedattention,” in *Symposium on Operating Systems Principles (SOSP)*, 2023.
- [13] A. Nag, S. Chakrabarti, A. Mukherjee, and N. Ganguly, “Efficient continual pre-training of llms for low-resource languages,” in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 3: Industry Track)*, 2025.
- [14] A. Vaswani, N. Shazeer, N. Parmar *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [15] T. B. Brown, B. Mann, N. Ryder *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [16] A. Yang, B. Yang, B. Hui *et al.*, “Qwen2 technical report,” *arXiv preprint*, 2024.
- [17] A. Yang, B. Yang, B. Zhang *et al.*, “Qwen2.5 technical report,” *arXiv preprint*, 2025.
- [18] A. Q. Jiang, A. Sablayrolles, A. Roux *et al.*, “Mixtral of experts,” *arXiv preprint*, 2024.
- [19] D. Guo, D. Yang, H. Zhang *et al.*, “Deepseek-r1 incentivizes reasoning in llms through reinforcement learning,” *arXiv preprint*, 2025.
- [20] A. Yang, A. Li, B. Yang *et al.*, “Qwen3 technical report,” *arXiv preprint*, 2025.
- [21] M. Chen, J. Tworek, H. Jun *et al.*, “Evaluating large language models trained on code,” *arXiv preprint*, 2021.
- [22] T. Schick, J. Dwivedi-Yu, R. Dessi *et al.*, “Toolformer: Language models can teach themselves to use tools,” in *Advances in Neural Information Processing Systems*, 2023.
- [23] Q. Wu, G. Bansal, J. Zhang *et al.*, “Autogen: Enabling next-gen llm applications via multi-agent conversation,” 2024.
- [24] J. Yang, C. Jimenez, A. Wettig *et al.*, “Swe-agent: Agent-computer interfaces enable automated software engineering,” in *Advances in Neural Information Processing Systems*, 2024.

TABLE XII: Full per-task accuracy (%) on InfiniteBench (Qwen2-7B-Instruct) for all nine SCR configurations and the baselines. Best per column in bold; R.Num/R.PK are saturated for the strong methods and left unbolded.

Method	Code.D	R.KV	En.Dia	Math.F	R.Num	R.PK
Qwen2-7B [16]	28.2	19.0	8.5	19.7	28.6	28.8
NTK [34]	24.9	59.8	7.5	27.7	97.5	99.2
SelfExtend [72]	8.1	0.0	5.0	2.3	0.0	0.0
StreamingLLM [48]	27.9	2.4	3.5	19.4	5.1	5.1
InfLLM [51]	27.4	5.4	7.5	24.0	72.2	70.3
TokenSelect [55]	26.7	91.0	10.0	24.0	100.0	100.0
$\theta=0.95, L_{\max}=1024$	27.6	94.0	10.0	23.7	100.0	100.0
$\theta=0.95, L_{\max}=2048$	26.1	96.2	10.0	23.4	100.0	100.0
$\theta=0.95, L_{\max}=4096$	24.9	98.0	6.0	22.6	100.0	100.0
$\theta=0.97, L_{\max}=1024$	27.0	94.0	11.0	23.1	100.0	100.0
$\theta=0.97, L_{\max}=2048$	27.9	96.2	9.0	22.9	100.0	100.0
$\theta=0.97, L_{\max}=4096$	25.8	98.0	8.0	22.6	100.0	100.0
$\theta=0.99, L_{\max}=1024$	28.8	94.0	11.5	23.1	100.0	100.0
$\theta=0.99, L_{\max}=2048$	26.7	96.6	10.0	23.7	100.0	100.0
$\theta=0.99, L_{\max}=4096$	25.8	98.0	7.0	23.4	100.0	100.0

TABLE XIII: Full per-task TTFT latency (s) on InfiniteBench for all nine SCR configurations. Best (lowest) per column in bold.

Method	Code.D	R.KV	En.Dia	Math.F	R.Num	R.PK
TokenSelect	8.8	13.2	7.7	8.7	9.4	9.3
$\theta=0.95, L_{\max}=1024$	6.7	9.9	6.0	6.6	7.1	7.0
$\theta=0.95, L_{\max}=2048$	5.9	8.7	5.3	5.9	6.3	6.3
$\theta=0.95, L_{\max}=4096$	5.7	8.3	5.1	5.7	6.1	6.0
$\theta=0.97, L_{\max}=1024$	6.7	9.8	6.0	6.6	7.0	7.0
$\theta=0.97, L_{\max}=2048$	6.1	8.7	5.4	5.9	6.4	6.4
$\theta=0.97, L_{\max}=4096$	5.8	8.3	5.1	5.7	6.1	6.1
$\theta=0.99, L_{\max}=1024$	7.3	9.9	6.2	6.7	7.4	7.5
$\theta=0.99, L_{\max}=2048$	6.8	8.7	5.7	5.9	6.7	6.6
$\theta=0.99, L_{\max}=4096$	6.7	8.3	5.7	5.7	6.3	6.3

- [25] Anthropic, “Claude code: An agentic cli for software engineering,” <https://www.anthropic.com/news/claude-3-7-sonnet>, 2025, accessed: 2026-05-18.
- [26] A. Amini, S. Gabriel, S. Lin *et al.*, “Mathqa: Towards interpretable math word problem solving with operation-based formalisms,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 2019.
- [27] K. Wang, H. Ren, A. Zhou *et al.*, “Mathcoder: Seamless code integration in llms for enhanced mathematical reasoning,” in *International Conference on Learning Representations*, 2024, pp. 5009–5042.
- [28] J. Macina, N. Daheim, S. P. Chowdhury *et al.*, “MathDial: A dialogue tutoring dataset with rich pedagogical properties grounded in math reasoning problems,” in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 5602–5621.
- [29] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” in *Advances in Neural Information Processing Systems Workshop*, 2014.
- [30] Y. Gu, L. Dong, F. Wei, and M. Huang, “Minillm: Knowledge distillation of large language models,” in *International Conference on Learning Representations*, 2024, pp. 32 694–32 717.
- [31] C.-Y. Hsieh, C.-L. Li, C.-K. Yeh *et al.*, “Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes,” in *Findings of the Association for Computational Linguistics: ACL 2023*, 2023, pp. 8003–8017.
- [32] S. Chen, S. Wong, L. Chen, and Y. Tian, “Extending context window of large language models via positional interpolation,” *arXiv preprint*, 2023.
- [33] J. Su, M. Ahmed, Y. Lu *et al.*, “Roformer: Enhanced transformer with rotary position embedding,” *Neurocomputing*, vol. 568, p. 127063, 2024.
- [34] “Neural tangent kernel (ntk) theory: Ntk-aware scaled rope allows llama models to have extended (8k+) context size without any fine-tuning and minimal perplexity degradation,” *Web*.
- [35] B. Peng, J. Quesnelle, H. Fan, and E. Shippole, “Yarn: Efficient context window extension of large language models,” in *International Conference on Learning Representations*, 2024, pp. 31 932–31 951.
- [36] A. Pal, D. Karkhanis, M. Roberts *et al.*, “Giraffe: Adventures in expanding context lengths in llms,” *arXiv preprint*, 2023.
- [37] G. Team, “Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context,” *arXiv preprint*, 2024.
- [38] “Post-training sparse attention with double sparsity,” *arXiv preprint*, 2024.
- [39] T. GLM, “Chatglm: A family of large language models from glm-130b to glm-4 all tools,” *arXiv preprint*, 2024.
- [40] M. Shoenybi, M. Patwary, R. Puri *et al.*, “Megatron-lm: Training multi-billion parameter language models using model parallelism,” *arXiv preprint*, 2020.
- [41] Z. Dai, Z. Yang, Y. Yang *et al.*, “Transformer-XL: Attentive language models beyond a fixed-length context,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.
- [42] J. W. Rae, A. Potapenko, S. M. Jayakumar, C. Hillier, and T. P.

- Lillicrap, “Compressive transformers for long-range sequence modelling,” in *International Conference on Learning Representations (ICLR)*, 2019, pp. 2978–2988.
- [43] A. Bulatov, Y. Kuratov, and M. Burtsev, “Recurrent memory transformer,” in *Advances in Neural Information Processing Systems*, 2022, pp. 11 079–11 091.
- [44] T. Munkhdalai, M. Faruqui, and S. Gopal, “Leave no context behind: Efficient infinite context transformers with infini-attention,” *arXiv preprint*, 2024.
- [45] Z. Wang, P. Ng, X. Ma, R. Nallapati, and B. Xiang, “Multi-passage BERT: A globally normalized BERT model for open-domain question answering,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 5878–5882.
- [46] J. Tian, D. Zheng, Y. Cheng *et al.*, “Untie the knots: An efficient data augmentation strategy for long-context pre-training in language models,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 1223–1242.
- [47] C. Han, Q. Wang, H. Peng *et al.*, “Lm-infinite: Zero-shot extreme length generalization for large language models,” in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2024, pp. 3991–4008.
- [48] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” in *International Conference on Learning Representations*, 2024, pp. 21 875–21 895.
- [49] Z. Zhang, Y. Sheng, T. Zhou *et al.*, “H2o: Heavy-hitter oracle for efficient generative inference of large language models,” in *Advances in Neural Information Processing Systems*, 2023, pp. 34 661–34 710.
- [50] Y. Li, Y. Huang, B. Yang *et al.*, “Snapkv: Llm knows what you are looking for before generation,” in *Advances in Neural Information Processing Systems*, 2024, pp. 22 947–22 970.
- [51] C. Xiao, P. Zhang, X. Han *et al.*, “Inflm: Training-free long-context extrapolation for llms with an efficient context memory,” in *Advances in Neural Information Processing Systems*, 2024, pp. 119 638–119 661.
- [52] J. Tang, Y. Zhao, K. Zhu *et al.*, “Quest: query-aware sparsity for efficient long-context llm inference,” in *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [53] J. Hao, Y. Zhu, T. Wang *et al.*, “Omnikv: Dynamic context selection for efficient long-context llms,” in *International Conference on Learning Representations*, 2025, pp. 87 443–87 464.
- [54] H. Jiang, Y. Li, C. Zhang *et al.*, “Minference 1.0: Accelerating pre-filling for long-context llms via dynamic sparse attention,” in *Advances in Neural Information Processing Systems*, 2024, pp. 52 481–52 515.
- [55] W. Wu, Z. Pan, C. Wang *et al.*, “TokenSelect: Efficient long-context inference and length extrapolation for LLMs via dynamic token-level KV cache selection,” in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025, pp. 21 264–21 281.
- [56] X.-P. Nguyen, M. Aljunied, S. Joty, and L. Bing, “Democratizing llms for low-resource languages by leveraging their english dominant abilities with linguistically-diverse prompts,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024.
- [57] Z. Ji, N. Lee, R. Frieske *et al.*, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [58] L. Huang, W. Yu, W. Ma *et al.*, “A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions,” *arXiv preprint arXiv:2311.05232*, 2023.
- [59] Y. Bang, S. Cahyawijaya, N. Lee *et al.*, “A multitask, multilingual, multimodal evaluation of ChatGPT on reasoning, hallucination, and interactivity,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2023, pp. 675–718.
- [60] S. Gao, C. Wang, C. Gao *et al.*, “The prompt alchemist: Automated llm-tailored prompt optimization for test case generation,” *arXiv preprint*, 2025.
- [61] B. Krause, E. Kahembwe, I. Murray, and S. Renals, “Dynamic evaluation of transformer language models,” *arXiv preprint*, 2019.
- [62] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems*, 2023, pp. 8634–8652.
- [63] M. Yuksekogonul, F. Bianchi, J. Boen *et al.*, “Optimizing generative ai by backpropagating language model feedback,” *Nature*, 2025.
- [64] L. A. Agrawal, S. Tan, D. Soylu *et al.*, “Gepa: Reflective prompt evolution can outperform reinforcement learning,” 2026.
- [65] Q. Zhang, C. Hu, S. Upasani *et al.*, “Agentic context engineering: Evolving contexts for self-improving language models,” in *The Fourteenth International Conference on Learning Representations*, 2026.
- [66] H. Zhou, Y. Chen, S. Guo *et al.*, “Memento: Fine-tuning llm agents without fine-tuning llms,” *arXiv preprint*, 2025.
- [67] X. Zhang, Y. Chen, S. Hu *et al.*, “ ∞ bench: Extending long context evaluation beyond 100k tokens,” pp. 15 262–15 277, 2024.
- [68] O. Skean, M. R. Arefin, D. Zhao *et al.*, “Layer by layer: Uncovering hidden representations in language models,” in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.
- [69] Z. Ye, L. Chen, R. Lai *et al.*, “FlashInfer: Efficient and customizable attention engine for LLM inference serving,” in *Proceedings of Machine Learning and Systems (MLSys)*, 2025.
- [70] L. Zheng, L. Yin, Z. Xie *et al.*, “SGLang: Efficient execution of structured language model programs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [71] C.-P. Hsieh, S. Sun, S. Kriman *et al.*, “Ruler: What’s the real context size of your long-context language models?” *arXiv preprint*, 2024.
- [72] H. Jin, X. Han, J. Yang *et al.*, “LLM maybe LongLM: SelfExtend LLM context window without tuning,” in *Proceedings of the 41st International Conference on Machine Learning*, 2024, pp. 22 099–22 114.